Institut für Parallele und Verteilte Systeme

Universität Stuttgart
Universitätsstraße 38
D–70569 Stuttgart

Masterarbeit

Connecting MetaConfigurator to the Semantic Web: RDF Authoring, Ontology Integration, and Knowledge Graph Support.

Mahdi Jafarkhani

Studiengang:	Computer Science
Prüfer/in:	Jun. Prof. Dr. rer.nat. Benjamin Uekermann
Betreuer/in:	Felix Neubauer, M.Sc.
Beginn am:	October 15, 2025
Beendet am:	April 15, 2026

Abstract

Scientific workflows increasingly produce structured JavaScript Object Notation (JSON) data that is easy to exchange but difficult to interpret semantically across systems. While JSON Schema supports structural validation, it does not provide native semantics for Linked Data integration. This thesis addresses that gap by extending MetaConfigurator, a schema-driven web application for structured data authoring, with Semantic Web capabilities.

The core contribution is an integrated Resource Description Framework (RDF) Authoring Panel that supports end-to-end semantic data workflows in a single User Interface. It enables researchers to connect existing traditional hierarchical or tabular research data to the Semantic Web by using AI-assisted RDF Mapping Language (RML) to transform these data into RDF. After transformation, the same panel supports refinement and modification of triples, SPARQL Protocol and RDF Query Language (SPARQL)-based exploration, interactive Knowledge Graph visualization, and export to common RDF serializations for import into existing knowledge stores.

The result is a unified tool that bridges traditional non-linked research data and Semantic Web infrastructures. The thesis demonstrates this tool on a real-world Metal-Organic Framework (MOF) synthesis application, showing how tabular research data can be transformed to RDF, iteratively improved, explored, visualized, and finally integrated into a target knowledge store.

Kurzfassung

Wissenschaftliche Workflows erzeugen zunehmend strukturierte JSON-Daten, die leicht austauschbar sind, jedoch systemübergreifend schwer semantisch zu interpretieren sind. Während JSON Schema die strukturelle Validierung unterstützt, bietet es keine nativen Semantiken für die Integration von Linked Data. Diese Arbeit schließt diese Lücke, indem sie MetaConfigurator, eine schema-gesteuerte Webanwendung zur Erstellung strukturierter Daten, um Semantic-Web-Funktionalitäten erweitert.

Der zentrale Beitrag ist ein integriertes RDF-Authoring-Panel, das End-to-End-Workflows für semantische Daten in einer einzigen Benutzeroberfläche unterstützt. Es ermöglicht Forschenden, bestehende traditionelle hierarchische oder tabellarische Forschungsdaten mit dem Semantic Web zu verknüpfen, indem KI-gestütztes RML verwendet wird, um diese Daten in RDF zu transformieren. Nach der Transformation unterstützt dasselbe Panel die Verfeinerung und Modifikation von Tripeln, die SPARQL-basierte Exploration, die interaktive Visualisierung von Wissensgraphen sowie den Export in gängige RDF-Serialisierungen zur Integration in bestehende Wissensspeicher.

Das Ergebnis ist ein einheitliches Werkzeug, das traditionelle, nicht verknüpfte Forschungsdaten mit Semantic-Web-Infrastrukturen verbindet. Die Arbeit demonstriert dieses Werkzeug anhand einer realen Anwendung zur MOF-Synthese und zeigt, wie tabellarische Forschungsdaten in RDF transformiert, iterativ verbessert, exploriert, visualisiert und schließlich in einen Ziel-Wissensspeicher integriert werden können.

Contents

List of Figures

List of Tables

Abbreviations

Notation	Description	Page List
AI	Artificial Intelligence	24, 26, 28, 30–32, 35, 43, 44, 54, 59, 60
CEDAR	Center for Expanded Data Annotation and Retrieval	27
ChEBI	Chemical Entities of Biological Interest	52, 59
CHMO	Chemical Methods Ontology	49, 52, 59
CSV	comma-separated values	20, 27, 32
ETL	extract, transform, load	28
GUI	graphical user interface	22, 37
IRI	Internationalized Resource Identifier	14–16, 25–27, 43
JSON	JavaScript Object Notation	2, 3, 10–12, 16, 20, 22, 23, 25–27, 29–32, 35, 37, 41, 43, 49, 52, 54, 59, 60
JSON-LD	JSON for Linked Data	16, 19, 20, 25–32, 35, 37, 39, 41–44, 49, 52, 54, 57, 59, 60
JSONPath	JSON Path	20
KG	Knowledge Graph	12, 16, 19, 20, 22, 25–27, 31, 47, 57–60
LLM	Large Language Model	54
MOF	Metal-Organic Framework	2, 3, 49, 54, 57, 59
OBI	Ontology for Biomedical Investigations	49, 52, 59
OWL	Web Ontology Language	16, 19, 20, 23, 27, 28

Notation	Description	Page List
QUDT	Quantities, Units, Dimensions, and Data Types	16, 19
R2RML	RDB to RDF Mapping Language	20, 29, 32
RDF	Resource Description Framework	2, 3, 10, 14, 16, 19, 20, 22, 23, 25–32, 35, 37, 39, 41–43, 49, 52, 54, 57, 59
RDFLib	RDFLib	28
RML	RDF Mapping Language	2, 3, 20, 25–27, 29–32, 35, 43, 44, 49, 52, 54, 59
SHACL	Shapes Constraint Language	16, 20, 28, 60
SPARQL	SPARQL Protocol and RDF Query Language	2, 3, 19, 25–28, 30, 31, 37, 43, 44, 52, 54, 57–59
SQL	Structured Query Language	20
UI	User Interface	59
W3C	World Wide Web Consortium	16, 20
XML	Extensible Markup Language	10, 20, 27, 32
XPath	XML Path Language	20
YAML	YAML Ain't Markup Language	27

1 Background and Related Work

The increasing adoption of computational experiments and simulation workflows in scientific research has resulted in large volumes of data [CBC+17; VKM+25; WAB+25]. In principle, such data can be modeled and validated directly with RDF-based standards [CWL14; W3C17]. In practice, however, many researchers still manage data in ad hoc artifacts such as spreadsheet files, and when machine-readable encodings are used, they are commonly represented in formats such as Extensible Markup Language (XML) or JSON, rather than as native RDF graphs [NBX+24; NEB+26].

While the Semantic Web promises improved interoperability, explicit semantics, and cross-dataset integration [BHL01; HBC+21; JHC+15], its adoption often faces a substantial entry barrier due to ontology engineering effort, identifier and vocabulary design, and mapping complexity [HBC+21; JHC+15]. To motivate a practical path across this gap, this chapter first introduces structured data representation and validation with JSON and JSON Schema. It then presents core Semantic Web concepts, and introduces MetaConfigurator as a tool for schema-driven data modeling and editing.

The chapter also reviews existing tools and approaches in areas such as ontology engineering, data transformation, and semantic data management. Although these solutions are powerful, they typically focus on specific tasks and are often used in isolation. As a result, users frequently have to switch between different tools to move from structured JSON data to semantically enriched RDF representations. This highlights a gap between data modeling and practical Semantic Web integration. By extending MetaConfigurator with support for semantic transformation, RDF authoring, querying, and visualization, this thesis aims to close this gap and provide a more integrated and accessible tool to work with semantically enriched data.

1.1 JSON and JSON Schema

1.1.1 JSON as a Data Representation Format

JSON is a lightweight data-interchange format widely used for representing structured data in web services, scientific workflows, and configuration systems. JSON encodes data as attribute-value pairs and ordered lists, making it both human-readable and machine-parseable. Due to its

simplicity and broad ecosystem support, JSON has become a de facto standard for structured data exchange across many domains [Bra17].

```json
{
  "parameters": [
    {
      "name": "element_size",
      "value": 0.025,
      "unit": "M"
    }
  ],
  "metrics": [
    {
      "name": "max_von_mises_stress",
      "value": 299791507.5586333,
      "unit": "MPa"
    }
  ]
}
```

Listing 1.1: Example JSON representation of simulation input parameters and results

The running example is derived from the benchmark concept presented by Unger et al. for validation and verification of simulation models for concrete structures [URR+26]. The benchmark context is a real finite-element simulation setting in which comparable test cases are executed across tools and mesh resolutions to evaluate reproducibility and implementation consistency. In such a setup, each run must be documented in a machine-readable form so that results can be compared, queried, and reproduced.

Listing 1.1 shows one such run record as two arrays: `parameters` for model inputs and `metrics` for computed outputs. Each entry follows the structure `name`, `value`, and `unit`. In this example, `element_size` is the characteristic mesh length for h-refinement (here $0.025\,\mathrm{m}$, where smaller values mean finer meshes), and `max_von_mises_stress` is the resulting maximum von Mises stress of the simulated specimen (approximately $299.8\,\mathrm{MPa}$) [URR+26].

While JSON provides a convenient serialization format, it does not inherently enforce structural constraints. Therefore, additional mechanisms are required to validate that JSON documents conform to an expected structure.

1.1.2 JSON Schema for Data Validation

JSON Schema provides a standardized mechanism for describing the expected structure and constraints of JSON documents. A JSON Schema defines properties, data types, required attributes, and validation rules for JSON instances. This allows automated validation of data. The JSON Schema specification defines a vocabulary for describing JSON documents and their validation rules [WAHD20].

Listing 1.2 presents a JSON Schema corresponding to the example JSON document in Listing 1.1. The schema mirrors the structure of the instance and ensures that the document contains the expected `parameters` and `metrics` arrays. Each entry in these arrays follows a common structure with the fields `name`, `value`, and `unit`, enforcing that every variable has a textual identifier, a numeric value, and an associated unit.

However, JSON Schema focuses primarily on structural validation and does not provide semantic meaning or interoperability with Knowledge Graph (KG) technologies. To address this limitation, semantic mapping techniques can be applied.

The schema in Listing 1.2 checks that simulation data has the expected format before it is processed further. It requires every parameter and metric to include a `name`, a numeric `value`, and a `unit`, and it blocks extra unexpected fields (`additionalProperties: false`). This avoids common problems such as missing units, wrong field names, or wrong data types when files are exchanged between tools. However, JSON Schema only checks structure. It does not describe the domain meaning of terms such as `element_size` or `max_von_mises_stress`. For that reason, we need to incorporate Semantic Web technologies to add explicit meaning and improve interoperability.

1.2 Semantic Web

The Semantic Web extends the World Wide Web by enabling data to be represented in a machine-readable, semantically rich form. This allows heterogeneous data to be interlinked, interpreted consistently, and queried across systems [BHL01].

1.2.1 Ontology: Modeling Knowledge Domains

Ontologies provide a formal specification of concepts and relationships within a domain of knowledge. One of the most widely cited and influential definitions is provided by Gruber [Gru93]:

> "An ontology is an explicit specification of a shared conceptualization."

In the context of scientific experiments, ontologies can define experiment parameters, measurement units, procedures, and results. For example, an ontology might define a class `PropertyValue` representing any measured property, a class `Method` representing experimental procedures, and relationships such as `hasParameter` and `investigates` to link methods to parameters and observed metrics.

Ontologies serve as the backbone for semantic data integration and reasoning. They allow data from heterogeneous sources to be interpreted consistently, facilitate interoperability, and enable advanced queries across datasets using formal reasoning engines.

```json
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Simulation Results",
  "type": "object",
  "properties": {
    "parameters": {
      "type": "array",
      "items": {
        "$ref": "#/$defs/variable"
      }
    },
    "metrics": {
      "type": "array",
      "items": {
        "$ref": "#/$defs/variable"
      }
    }
  },
  "required": ["parameters", "metrics"],
  "additionalProperties": false,
  "$defs": {
    "variable": {
      "type": "object",
      "properties": {
        "name": {
          "type": "string",
          "description": "Name of the parameter or metric"
        },
        "value": {
          "type": "number",
          "description": "Numeric value of the parameter or metric"
        },
        "unit": {
          "type": "string",
          "description": "Unit symbol (e.g., m, MPa)"
        }
      },
      "required": ["name", "value", "unit"],
      "additionalProperties": false
    }
  }
}
```

Listing 1.2: JSON Schema validating the simulation result for JSON document in Listing 1.1

1.2.2 RDF: Resource Description Framework

RDF is the core data model of the Semantic Web. It represents information as triples of *subject*, *predicate*, and *object*, which together form a graph structure [CWL14]. This model provides a common, tool-independent way to express entities and relationships across datasets.

Formally, a RDF triple is a 3-tuple [CWL14]:

$$(s, p, o) \in (U \cup B) \times U \times (U \cup B \cup L)$$

where:

- U is the set of all Internationalized Resource Identifiers (IRIs),
- B is the set of blank nodes (anonymous resources),
- L is the set of RDF literals (e.g., strings, numbers, dates),
- $s \in U \cup B$ is the subject,
- $p \in U$ is the predicate (also called property),
- $o \in U \cup B \cup L$ is the object.

A RDF graph G is then defined as a finite set of such triples:

$$G = \{(s_i, p_i, o_i) \mid i = 1, \ldots, n\}, \quad s_i \in U \cup B,\ p_i \in U,\ o_i \in U \cup B \cup L$$

```
<rdf:RDF
 xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
 xmlns:schema="https://schema.org/">
    <schema:PropertyValue rdf:about="https://www.example.org/variable_element_size">
        <schema:name>element_size</schema:name>
        <schema:unitCode rdf:resource="http://qudt.org/vocab/unit/M"/>
        <schema:value rdf:datatype="http://www.w3.org/2001/XMLSchema#double">2.5E-2</schema:value>
    </schema:PropertyValue>
</rdf:RDF>
```

Listing 1.3: RDF/XML representation of parameter `element_size` from Listing 1.1

The RDF model in Listing 1.3 adds an extra layer of meaning to the same data in Listing 1.1. RDF/XML models the data as explicit graph statements instead of nested key-value objects, as depicted in Figures 1.1 and 1.2[1]. In JSON, fields such as `name`, `unit`, and `value` are interpreted mainly by application logic, whereas in RDF the subject (`variable_element_size`) is connected through well-defined predicates (`schema:name`, `schema:unitCode`, `schema:value`) to typed objects,

[1] Visualization and Table view were generated with ISSemantic: `https://issemantic.net/`

including links to specific ontologies such as QUDT for units. This makes the representation less format-specific and more interoperable, because the meaning is encoded directly in standard IRI and data types rather than implied by local JSON structure.

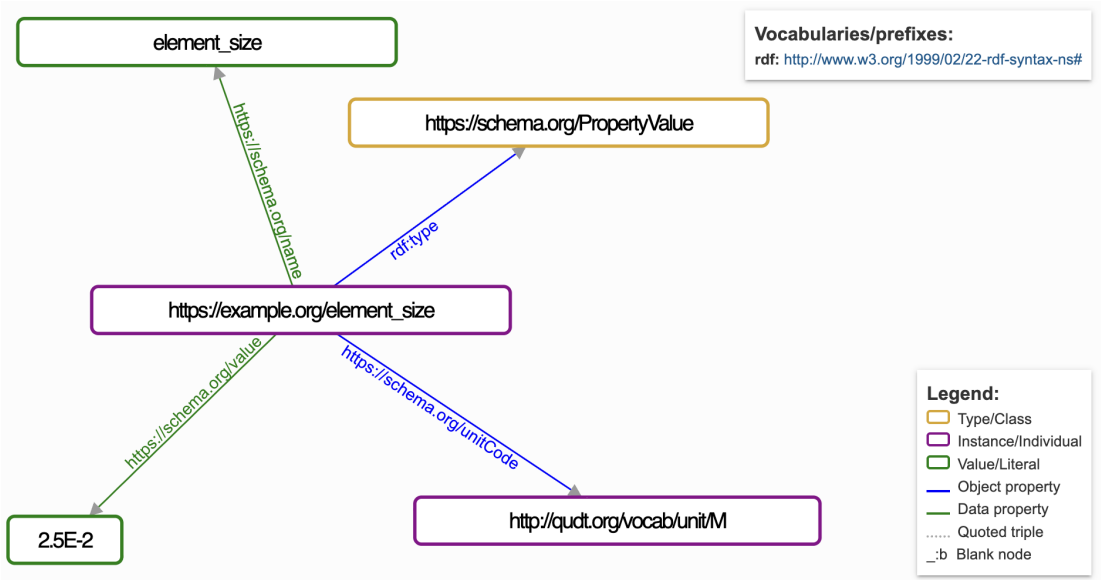


Figure 1.1: Graph view of the RDF representation in Listing 1.3

Subject	Predicate	Object
https://www.example.org/variable_element_size	rdf:type	https://schema.org/PropertyValue
https://www.example.org/variable_element_size	https://schema.org/name	element_size
https://www.example.org/variable_element_size	https://schema.org/unitCode	http://qudt.org/vocab/unit/M
https://www.example.org/variable_element_size	https://schema.org/value	2.5E-2

Figure 1.2: Table view of the RDF representation in Listing 1.3

1.2.3 Semantic Data Representation using JSON-LD

JSON for Linked Data (JSON-LD) is another serialization format for RDF graphs, but expressed in JSON syntax [SKL14]. While RDF/XML (Listing 1.3) uses XML tags, JSON-LD represents the same graph concepts in a JSON-friendly structure that is often easier to integrate into existing data pipelines and web-based tools.

A typical JSON-LD document has two key structural components:

- `@context`: This section defines the mapping between JSON keys and ontology IRIs or vocabularies. It provides semantic meaning to each field.
- `@graph`: This array contains the actual data entities, each represented as a node with `@id`, `@type`, and associated properties. Relationships between entities are expressed using references to other `@id` values, forming a connected graph structure.

Listing 1.4 shows a complete JSON-LD representation corresponding to the example in Listing 1.1. The `@context` defines prefixes such as `schema` and `unit`, while the `@graph` contains nodes representing the simulation run, the `element_size` parameter, and the stress metric.

By modeling the experimental data from Listing 1.1 in JSON-LD and explicitly defining its semantics, the data becomes linked to the Semantic Web and can be integrated into a KG. In this representation, `element_size` is modeled as a `schema:PropertyValue` resource with a typed numeric value and an explicit unit IRI, just as in RDF/XML. Compared with Listing 1.3, the semantics are equivalent, but the syntax remains JSON-based.

1.2.4 SHACL: Shapes Constraint Language

The Shapes Constraint Language (SHACL) is a World Wide Web Consortium (W3C) standard for validating RDF graphs against a set of structural and semantic constraints [W3C17]. SHACL performs validation based on explicitly defined "shapes" that describe the expected properties, data types, and cardinality of RDF nodes. While these shapes can be designed to reflect or align with domain ontologies, SHACL validation itself is independent of ontology languages such as Web Ontology Language (OWL), which are primarily used for logical reasoning rather than constraint checking.

For example, consider the simulation parameter `element_size` in the JSON-LD example (Listing 1.4). Using SHACL, we can define a shape that enforces that every `schema:PropertyValue` representing an experiment parameter must have a numeric `schema:value` and a `schema:unitCode` pointing to a valid unit from the Quantities, Units, Dimensions, and Data Types (QUDT) ontology:

```json
{
    "@context": {
        "m4i": "http://w3id.org/nfdi4ing/metadata4ing#",
        "schema": "https://schema.org/",
        "unit": "http://qudt.org/vocab/unit/",
        "local": "https://www.example.org/"
    },
    "@graph": [
        {
            "@id": "local:run_fenics_simulation",
            "@type": "m4i:Method",
            "m4i:hasParameter": {
                "@id": "local:variable_element_size"
            },
            "m4i:investigates": {
                "@id": "local:variable_max_von_mises_stress"
            }
        },
        {
            "@id": "local:variable_element_size",
            "@type": "schema:PropertyValue",
            "schema:name": "element_size",
            "schema:unitCode": {
                "@id": "unit:M"
            },
            "schema:value": {
                "@type": "http://www.w3.org/2001/XMLSchema#double",
                "@value": "2.5E-2"
            }
        },
        {
            "@id": "local:variable_max_von_mises_stress",
            "@type": "schema:PropertyValue",
            "schema:name": "max_von_mises_stress",
            "schema:unitCode": {
                "@id": "unit:MPa"
            },
            "schema:value": {
                "@type": "http://www.w3.org/2001/XMLSchema#double",
                "@value": "2.997915075586333E8"
            }
        }
    ]
}
```

Listing 1.4: JSON-LD representation of the simulation result presented in Listing 1.1

```
@prefix sh: <http://www.w3.org/ns/shacl#> .
@prefix schema: <https://schema.org/> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix unit: <http://qudt.org/vocab/unit/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix local: <https://www.example.org/> .

local:PropertyValueShape
    a sh:NodeShape ;
    sh:targetClass schema:PropertyValue ;

    # The variable name label
    sh:property [
        sh:path rdfs:label ;
        sh:datatype xsd:string ;
        sh:minCount 1 ;
        sh:maxCount 1 ;
    ] ;

    # The numeric value of the parameter or metric
    sh:property [
        sh:path schema:value ;
        sh:datatype xsd:double ;
        sh:minCount 1 ;
        sh:maxCount 1 ;
    ] ;

    # The unit, as a named QUDT IRI
    sh:property [
        sh:path schema:unitCode ;
        sh:nodeKind sh:IRI ;
        sh:minCount 1 ;
        sh:maxCount 1 ;
    ] .
```

Listing 1.5: SHACL shape for validating simulation parameters

By applying this shape, RDF data can be automatically checked for compliance with the expected structure and semantics, preventing errors or inconsistencies before integration into a KG.

1.2.5 SPARQL: Querying Semantic Data

SPARQL is the standard query language for RDF datasets [PS13], supporting retrieval, filtering, and aggregation of data stored in KGs. Queries are expressed in terms of triples, similar to RDF statements.

Listing 1.6 presents an SPARQL example using the JSON-LD simulation data, where all experiment parameters and metrics are retrieved together with their corresponding values and units.

```
PREFIX schema: <https://schema.org/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX unit: <http://qudt.org/vocab/unit/>
PREFIX local: <https://www.example.org/>

SELECT ?parameter ?value ?unit
WHERE {
    ?param a schema:PropertyValue ;
           rdfs:label ?parameter ;
           schema:value ?value ;
           schema:unitCode ?unit .
}
```

Listing 1.6: SPARQL query retrieving parameter values and units

This query retrieves the parameters `element_size` and `max_von_mises_stress` together with their corresponding numeric values and associated QUDT units (`unit:M` and `unit:MPa`). The result reflects the structure of the JSON-LD representation, in which both parameters and metrics are modeled as `schema:PropertyValue` resources.

This example illustrates how semantic annotations enable structured and consistent access to data. SPARQL further provides mechanisms for querying RDF graphs, including filtering and combining data from multiple sources, which supports the analysis of semantically enriched simulation data.

1.2.6 OWL: Modeling Rich Semantics and Logical Rules

While RDF provides the graph structure and SPARQL enables querying, OWL adds a formal layer for expressing richer domain knowledge [W3C12a; W3C12b]. With OWL, one can define class hierarchies, equivalence and disjointness of classes, and logical constraints on properties

(e.g., transitive, symmetric, or cardinality-based restrictions). This allows data to be interpreted not only as stored triples, but also through the rules encoded in the ontology.

In the context of the running simulation example, OWL can make implicit knowledge explicit. Assume the ontology defines `m4i:SimulationMethod` as a subclass of `m4i:Method`, and constrains `m4i:hasParameter` to point only to `schema:PropertyValue` resources. If the data states that `local:run_fenics_simulation` is a `m4i:SimulationMethod` and links it via `m4i:hasParameter` to `local:variable_element_size`, an OWL reasoner can infer that `local:run_fenics_simulation` is also a `m4i:Method` and that `local:variable_element_size` is a `schema:PropertyValue`, even when these types are not asserted explicitly.

Compared with SHACL, which is mainly used to validate whether data satisfies predefined constraints, OWL focuses on deriving additional knowledge from formally defined semantics. In practice, both are complementary: SHACL helps ensure data quality, while OWL supports richer interpretation and inference in a KG.

1.2.7 RML for Mapping JSON to RDF

RML is a declarative language for transforming heterogeneous data sources into RDF representations. RML extends the W3C-standardized RDB to RDF Mapping Language (R2RML) language and supports mappings from JSON, XML, comma-separated values (CSV), and relational databases into RDF datasets [DVC+14].

RML mappings are themselves RDF graphs specifying how elements from a source data structure are transformed into RDF triples. Each mapping defines:

- **Logical source:** the data source and reference formulation (e.g., JSON Path (JSONPath), XML Path Language (XPath), Structured Query Language (SQL) query),
- **Subject map:** rules to generate the *subject* of triples,
- **Predicate-object maps:** rules to generate the *predicate* and *object* for triples.

Listing 1.7 shows part of an example RML mapping that converts the JSON simulation output from Listing 1.1 into the JSON-LD structure presented in Listing 1.4.

As one concrete example, `<#ParameterMapping>` iterates over the array `$.parameters[*]` in the JSON document. For each entry, it creates an RDF resource with an IRI constructed from the `name` field (e.g., `local:variable_element_size`) and assigns it the type `schema:PropertyValue`. The JSON attribute `name` is mapped to `rdfs:label`, while the numeric field `value` is mapped to `schema:value`. The unit is converted into a QUDT unit IRI by constructing the resource `http://qudt.org/vocab/unit/{unit}` and assigning it via `schema:unitCode`.

```
@prefix rr: <http://www.w3.org/ns/r2rml#> .
@prefix rml: <http://semweb.mmlab.be/ns/rml#> .
@prefix ql: <http://semweb.mmlab.be/ns/ql#> .
@prefix m4i: <http://w3id.org/nfdi4ing/metadata4ing#> .
@prefix schema: <https://schema.org/> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix unit: <http://qudt.org/vocab/unit/> .
@prefix local: <https://www.example.org/> .

<#RunFenicsSimulation>
  rml:logicalSource [
    rml:source "Data.json" ;
    rml:referenceFormulation ql:JSONPath ;
    rml:iterator "$"
  ] ;

  rr:subjectMap [
    rr:template "https://www.example.org/run_fenics_simulation" ;
    rr:class m4i:Method
  ] ;

  rr:predicateObjectMap [
    rr:predicate m4i:hasParameter ;
    rr:objectMap [ rr:parentTriplesMap <#ParameterMapping> ]
  ] ;

  rr:predicateObjectMap [
    rr:predicate m4i:investigates ;
    rr:objectMap [ rr:parentTriplesMap <#MetricMapping> ]
  ] .

<#ParameterMapping>
  rml:logicalSource [
    rml:source "Data.json" ;
    rml:referenceFormulation ql:JSONPath ;
    rml:iterator "$.parameters[*]"
  ] ;

  rr:subjectMap [
    rr:template "https://www.example.org/variable_{name}" ;
    rr:class schema:PropertyValue
  ] ;

  rr:predicateObjectMap [
    rr:predicate rdfs:label ;
    rr:objectMap [ rml:reference "name" ]
  ] ;

  rr:predicateObjectMap [
    rr:predicate schema:value ;
    rr:objectMap [ rml:reference "value" ]
  ] ;

  rr:predicateObjectMap [
    rr:predicate schema:unitCode ;
    rr:objectMap [
      rr:template "http://qudt.org/vocab/unit/{unit}" ;
      rr:termType rr:IRI
    ]
  ] .
...
```

Listing 1.7: Subset of a RML mapping which converts JSON data in Listing 1.1 to JSON-LD in Listing 1.4

Through this mapping process, structured simulation outputs are automatically converted into RDF triples forming a semantically enriched KG.

1.3 MetaConfigurator for Schema-Driven Data Modeling

The technologies discussed in the previous sections enable structured, validated, and semantically enriched data representations. However, creating and editing such structured data manually can be challenging, particularly for users who are not familiar with textual formats such as JSON or with schema-based validation mechanisms. To address this challenge, tools that support schema-driven data modeling and editing are required.

MetaConfigurator[2] is an open-source web application designed to simplify the creation, editing, and management of structured data files and their accompanying schemas [NBX+24; NPU25]. The tool generates graphical user interfaces (GUIs) dynamically based on a provided schema, allowing users to interact with structured data through intuitive forms rather than manually editing raw text files. By leveraging JSON Schema as the underlying model description, MetaConfigurator enables consistent data validation and guided data entry.

In practice, MetaConfigurator is centered around two main working tabs, **Data** and **Schema**, while **Settings** is used for global configuration [NPU25].

1.3.1 Core Features

- **Visual Schema Editor:** Users can create and modify JSON schemas using a graphical interface, enabling intuitive construction of data models without manually writing schema definitions. Figure 1.4 depicts an example of schema editing in MetaConfigurator, showing the JSON Schema from Listing 1.2. The left panel shows the raw JSON Schema, the middle panel provides a diagram view for editing, and the right panel offers a structured form for schema modification.
- **Simplified and Advanced Schema Modes:** To reduce complexity while preserving expressiveness, MetaConfigurator provides configurable schema-editing modes. A simplified mode hides advanced JSON Schema options, while an advanced mode exposes the full feature set. This behavior is implemented through a meta-schema-builder approach that dynamically derives the editor schema from user settings [NPU25].
- **Schema-Driven Form Generation:** Based on a given JSON Schema, MetaConfigurator automatically generates a graphical form for editing instance data. This ensures that all entered data conforms to the defined structure and constraints.

[2] `https://www.metaconfigurator.org`

- **Schema Validation Support:** During data editing, MetaConfigurator validates instance data against the provided JSON Schema, ensuring structural correctness and preventing invalid data entries.
- **Schema Visualization and Navigation:** Complex schemas can be explored using a tree-like or diagram-based representation, helping users understand hierarchical structures and relationships between properties. The 2025 extension introduces an interactive diagram-like schema view with synchronized editing actions (e.g., renaming and adding properties), while advanced constructs such as compositions and conditionals are primarily visualized rather than fully edited graphically [NPU25].
- **CSV Import and Schema Inference:** Tabular data can be imported from CSV/Excel-oriented workflows into JSON, with automatic initial schema inference to reduce manual modeling effort [NPU25].
- **Snapshot Sharing and URL-Based Collaboration:** MetaConfigurator supports sharing complete editor states (data, schema, settings) through shareable links and snapshot storage, enabling reproducible collaboration between users [NPU25].
- **Limited Ontology Integration:** While the tool provides support for JSON-LD by recognizing `@context` and `@id` fields, its ontology integration capabilities remain limited. In its current form, a field can be annotated as JSON-LD and connected to a SPARQL endpoint that may be used to retrieve lists of ontology concepts, such as OWL classes or RDF properties, to support basic auto-completion during data modeling.

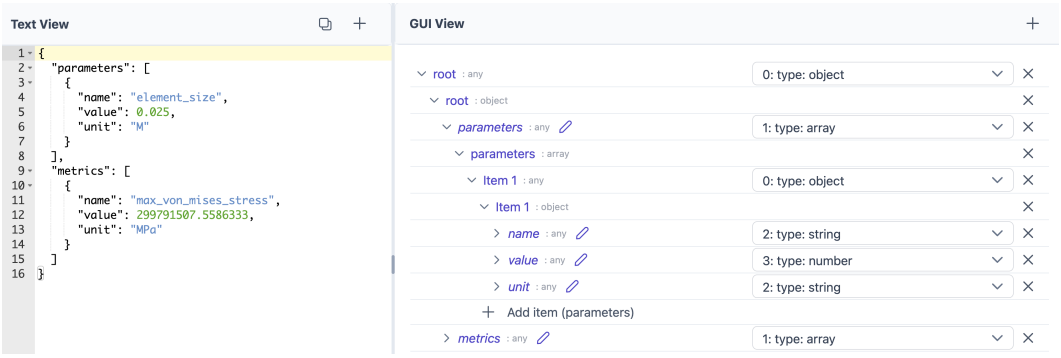


Figure 1.3: A snapshot of MetaConfigurator's data editing interface, showing the JSON instance from Listing 1.1

This combination of views gives users a continuous workflow from low-level text editing to guided schema-driven interaction, and reduces the effort needed to model, validate, and maintain structured data across different domains [NBX+24; NPU25].

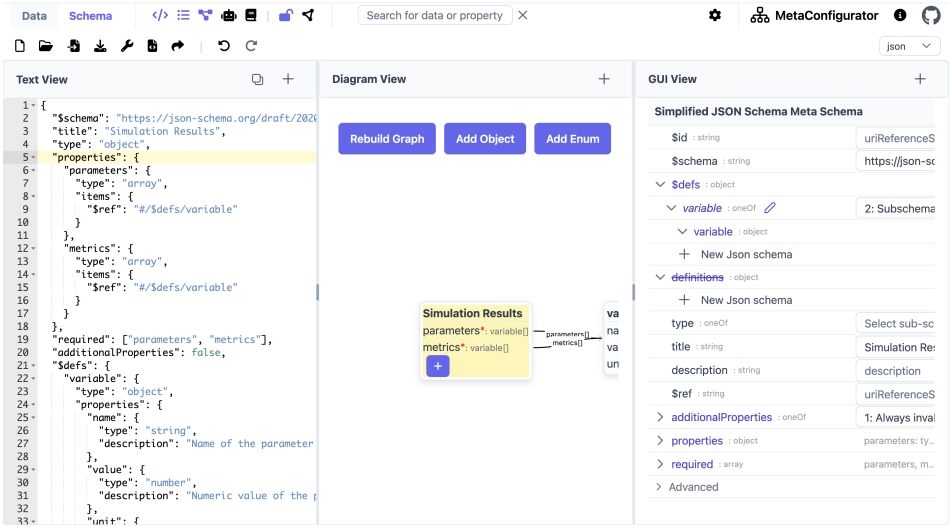


Figure 1.4: A snapshot of MetaConfigurator's schema editing interface, showing the JSON Schema from Listing 1.2

1.3.2 AI-Assisted Schema Engineering and Mapping

Recent work extends MetaConfigurator with Artificial Intelligence (AI)-assisted support for schema creation, modification, and mapping from natural-language user input [NUP25]. The approach combines large language models with deterministic safeguards to improve reliability and usability.

For schema authoring, MetaConfigurator provides a chat-like interface where users describe their intent in natural language. The system then performs prompt construction and context management automatically, including role instructions, task-specific formatting constraints, and relevant schema context. Generated results are directly validated and visualized in the schema editor, and users remain in control through a manual validation workflow where generated output can be reviewed and refined before acceptance.

For data transformation, the system generates mapping expressions from user input and executes these mappings deterministically. In the presented implementation, JSONata expressions are generated from an input document and a target schema, then validated and executed in a controlled pipeline. This separates AI-based generation from rule execution and supports scalable transformation workflows across heterogeneous source formats.

1.3.3 Limitations for Semantic Web Integration

Despite its strong support for schema-driven data modeling, MetaConfigurator currently provides only limited support for Semantic Web technologies: while the platform enables the creation and validation of structured data using JSON and JSON Schema, it has limited support for Linked Data representations. Currently, MetaConfigurator treats JSON documents primarily as structured configuration data and does not interpret or expose their underlying RDF semantics.

Several limitations arise from this restriction:

1. **Lack of JSON-LD context authoring:** Users cannot directly author or edit JSON-LD contexts (e.g., the `@context` section that defines mappings between JSON keys and ontology IRIs). Without such support, linking data elements to ontologies requires manual editing outside the tool.

   Users can define a JSON-LD context when JSON-LD support is enabled; however, this functionality is limited and currently only available within the schema editor. The `@context` section, which defines mappings between JSON keys and ontology IRIs, is not exposed in the data editing view. As a result, working with or adjusting context definitions remains less accessible during instance-level data authoring.

2. **No direct inspection or editing of RDF triples:** MetaConfigurator does not provide mechanisms for inspecting or manipulating RDF triples derived from the underlying JSON data. As a result, users cannot directly view the semantic graph structure implied by JSON-LD documents.

3. **Lack of transformation from JSON to RDF:** The platform lacks built-in mechanisms for transforming conventional JSON documents into semantic representations. As discussed in Section 1.2.7, technologies such as RML enable declarative transformations from JSON to RDF, but such transformations are not currently integrated into MetaConfigurator's workflow.

4. **Lack of semantic querying capabilities:** MetaConfigurator does not support semantic querying of RDF data. As described in Section 1.2.5, SPARQL provides a standard mechanism for querying RDF graphs and retrieving structured information from KGs.

5. **No visualization of RDF KGs:** KGs are often easier to understand when represented visually as nodes and relationships. However, the current platform does not provide mechanisms to visually explore RDF triples or inspect the structure of the resulting KG.

To address these limitations, this thesis extends MetaConfigurator with an **RDF Authoring Panel** that integrates Semantic Web technologies directly into the schema-driven editing environment. The main contributions of this thesis include:

1. **Transformation of JSON documents to JSON-LD:** The system enables the transformation of conventional JSON documents into JSON-LD using RML mappings. This allows structured JSON data created within MetaConfigurator to be converted into RDF triples that can be integrated into Semantic Web infrastructures.
2. **RDF authoring and editing support:** The RDF Authoring Panel allows users to directly inspect and edit RDF data. This includes editing the JSON-LD `@context` as well as creating and modifying RDF triples within the interface.
3. **Ontology-aware IRI auto-completion:** The system integrates ontology lookup services that provide auto-completion for IRIs when authoring RDF triples. This facilitates the reuse of existing ontologies and promotes semantic interoperability.
4. **SPARQL querying support:** The extension introduces the ability to query RDF data using SPARQL. This allows users to retrieve and analyze information from the generated KG directly within the MetaConfigurator environment.
5. **Interactive KG visualization:** The RDF triples generated from JSON-LD can be visualized as an interactive graph representation. This visualization enables users to explore entities and relationships within the KG and understand the semantic structure of the data more easily.
6. **AI-assisted SPARQL and RML authoring from user hints:** Building on MetaConfigurator's existing AI-assisted schema capabilities, this thesis introduces AI-assisted generation of SPARQL queries and RML mappings from user-provided hints. Users can describe their intent in natural language and receive draft SPARQL queries and RML mapping definitions that can be reviewed and refined in the RDF Authoring Panel.

By integrating these capabilities into MetaConfigurator, the platform evolves from a schema-based configuration editor into a tool that supports the full workflow of creating, transforming, querying, and exploring semantically enriched data.

1.4 Related Work

Related work in this area spans several mature but often separate tool families: ontology engineering platforms, transformation tools that convert source data into RDF, schema/metadata modeling environments for structured data, and libraries or engines for querying and validation. These families provide strong support for individual tasks, but integrated workflows that combine guided JSON authoring, semantic transformation, RDF curation, querying, and AI-assisted support in a single user-facing environment are still uncommon.

1.4.1 Ontology Engineering and KG Platforms

Protégé and WebProtégé are widely used for ontology engineering and collaborative ontology curation [CBI; HGN+14]. Their strength is formal ontology modeling, but this also implies a relatively high entry barrier: users need to understand RDF/OWL concepts, IRIs, and KG thinking before they can work productively [HBC+21; JHC+15]. In many practical settings, however, data already exists in traditional non-linked formats such as JSON, XML, CSV, or YAML Ain't Markup Language (YAML), and teams do not start from native RDF. MetaConfigurator addresses this gap by supporting import and schema-driven editing of such existing formats [NBX+24; NPU25]; the RDF Authoring Panel could provide the next step as a bridge from conventional structured data to semantic RDF/KG workflows.

Apache Jena provides a mature programmatic stack for RDF processing, reasoning, and SPARQL querying [Fou]. GraphDB focuses on RDF storage and query execution in production-oriented KG settings [Ont]. These systems are strong back-end components, but typically require separate front-end layers for guided data entry and authoring workflows.

1.4.2 Schema and Metadata Modeling for Structured Data

LinkML is a schema-centric tool in this context. It follows a model-driven approach in which a single high-level schema can be used to generate multiple artifacts, including validation schemas and semantic representations such as JSON-LD and OWL [MSU+21]. This is highly valuable for maintaining consistency across data models and downstream implementations. At the same time, effective use of LinkML still requires substantial conceptual understanding of schema design, semantic modeling decisions, and artifact generation workflows, which can be challenging for domain users without prior Semantic Web experience.

The Center for Expanded Data Annotation and Retrieval (CEDAR) Workbench also improves metadata quality through ontology-assisted authoring in scientific contexts [GOM+19]. However, as with other advanced modeling environments, the practical starting point in many projects is not a clean semantic model, but existing non-linked datasets in operational formats such as JSON, XML, CSV, or YAML. For these scenarios, a bridge approach is needed: first support users in editing and structuring their existing data, then map it into RDF. This is why declarative mapping approaches such as RML are particularly suitable in our setting (Section 1.2.7), and why MetaConfigurator is positioned as a gradual path from conventional structured data toward semantic KG workflows rather than assuming that users begin directly with native RDF.

1.4.3 Transformation from Source Data to RDF

Karma, OpenRefine (with RDF extensions), and SPARQL-Generate support transformation from heterogeneous sources to RDF [CG; ISI20; Proa]. These tools are effective for extract, transform, load (ETL)-oriented conversion pipelines and are strong choices when the main goal is mapping source records into triples. However, they are typically centered on the transformation step itself. In contrast, our approach targets a broader user workflow: preparing existing non-linked data, transforming it to RDF, and then continuing with semantic authoring, inspection, and querying in one environment.

1.4.4 Querying, Validation, and Programmatic RDF Processing

RDF validation and querying standards such as SHACL and SPARQL are well established [PS13; W3C17]. For lightweight experimentation, JSON-LD Playground provides quick inspection of contexts and RDF output [W3C23]. RDFLib (RDFLib) and Redland provide low-level programmatic RDF processing [Prob; Proc]. These tools are important ecosystem building blocks but are not, by themselves, unified authoring environments.

1.4.5 Comparison of Semantic Web Editing Tools

Table 1.1 compares a subset of Semantic Web editing tools across capabilities relevant to this thesis: RDF authoring, ontology integration, source-to-RDF transformation, RDF storage/query, visualization, and AI assistance.

1.5 Research Gap and Positioning of This Thesis

Although these tools provide strong support for individual Semantic Web tasks, several limitations remain for practical, user-facing scientific data workflows:

- **Fragmented workflows:** Modeling, mapping, querying, and visualization are often distributed across multiple tools, requiring users to switch between different environments to complete a single data integration task [HBC+21].
- **High technical entry barrier:** Effective use often requires advanced knowledge of OWL, RDF, SPARQL, and mapping languages, which limits accessibility for domain scientists who are not Semantic Web experts [JHC+15].

Table 1.1: Comparison of tools supporting RDF authoring and Semantic Web integration

Tool / Framework	RDF Authoring	Ontology Integration	Source-to-RDF Transform.	RDF Storage / Query	Visualization	AI Assistance
Protégé	✓	✓	–	–	✓	–
WebProtégé	✓	✓	–	–	Limited	–
Apache Jena	✓	✓	Limited	✓	–	–
LinkML	–	✓	Partial	–	–	–
SemTK [Sem]	✓	✓	Partial	✓	✓	–
OpenRefine (RDF)	Limited	–	✓	–	–	–
SPARQL-Generate	–	–	✓	–	–	–
JSON-LD Playground	Limited	–	Limited	–	Limited	–
RDFLib	✓	Limited	–	Partial	–	–
Redland RDF	✓	Limited	–	Partial	–	–

Table 1.2: Rating scale used in Table 1.1

Symbol / Term	**Meaning**
✓	Native, first-class support
Partial	Substantial but scope-limited support (often programmatic rather than end-user-focused)
Limited	Basic or indirect support (e.g., plugin-based, narrow subset, or lightweight/read-only capability)
–	Not a core capability

- **Gap between structured JSON editing and RDF transformation:** In practice, much scientific data is authored and maintained in hierarchical formats such as JSON. While standards such as JSON-LD provide a bridge to linked data [SKL14], moving from conventional JSON structures to RDF still often requires separate mapping artifacts and additional tools (e.g., R2RML/RML) [DSC12; DVC+14].

- **Limited end-to-end support in one UI:** Most tools focus on a single task—such as ontology editing, mapping definition, or querying—rather than supporting the entire workflow from data authoring and transformation to validation, querying, and graph exploration in a unified environment.
- **Limited AI support in semantic authoring workflows:** Despite recent advances in generative AI, assistance for generating artifacts such as SPARQL queries or RML mappings from natural-language descriptions of user intent is still rarely integrated into Semantic Web authoring tools.

This thesis addresses these gaps by extending MetaConfigurator from a schema-guided data modeling tool into an integrated RDF authoring environment. In particular, it supports a continuous workflow from JSON authoring to JSON-LD/RDF transformation, ontology-aware triple editing, integrated SPARQL querying, graph visualization, and AI-assisted drafting of SPARQL queries and RML mappings.

Chapter 2 builds on this background by translating these requirements into concrete architectural and implementation decisions for the RDF authoring extension in MetaConfigurator.

2 Design and Implementation

This chapter describes how the proposed Semantic Web extensions are realized in MetaConfigurator. This enables users to move from conventional structured data to semantically enriched JSON-LD/RDF, author and inspect triples, execute SPARQL queries, and explore resulting KGs within one coherent interface. The chapter is structured along this workflow: Section 2.1 introduces the RDF Authoring Panel, including RML-based transformation support, panel composition, synchronization between text and triple views, and linked selection behavior; Section 2.2 presents query authoring and execution with SPARQL, including AI-assisted query drafting; and Section 2.3 describes interactive KG visualization and editing support for graph-centric exploration.

2.1 RDF Authoring Panel

The RDF authoring functionality is implemented as a dedicated, separate panel in MetaConfigurator, located in `meta_configurator/src/components/panels/rdf`. This design keeps semantic authoring concerns modular while still integrating with the existing panel configurations, path selection, and data editing flow of MetaConfigurator [NBX+24]. At runtime, the entry component `RdfPanel.vue` validates whether the current data can be treated as JSON-LD and provides immediate warnings for missing `@context`, invalid syntax, and non-JSON-LD input.

More generally, panel integration in MetaConfigurator follows a decoupled architecture: panels do not update each other directly, but communicate through a central reactive store as the single source of truth. Each panel writes data/schema updates to the store and watches store state for incoming changes. The same mechanism is used for path selection, so selection updates made in one view are propagated through the store and reflected consistently in the others.

Figure 2.1 shows a snapshot of the RDF panel in action, loaded with the JSON example from 1.1. Since the data is provided in JSON format, the panel indicates that it must first be converted to JSON-LD before further processing. The user can proceed in two ways: either by directly supplying a JSON-LD document in the Text View, or by using the JSON-to-JSON-LD conversion tool discussed in Section 2.1.1.

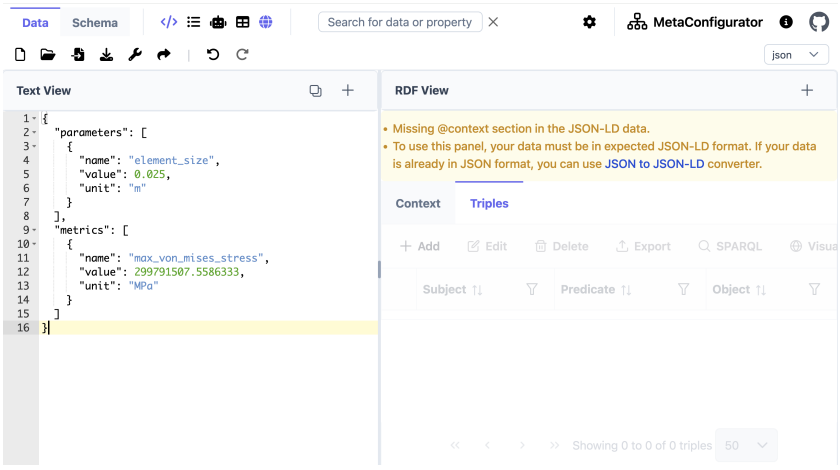


Figure 2.1: A snapshot of MetaConfigurator's RDF panel.

2.1.1 RML Mapping Dialog

If the input data is not already expressed as JSON-LD, the RDF panel provides a conversion tool that enables users to transform JSON data into RDF-compatible JSON-LD using RML mappings. RML extends the R2RML mapping language and allows the transformation of heterogeneous data sources such as JSON, CSV, or XML into RDF graphs [DVC+14], as discussed in Section 1.2.7. Figure 2.2 illustrates the dialog used to define such mappings.

The dialog allows users to either provide an existing RML mapping or generate one automatically with AI assistance. In the latter case, the user supplies mapping hints that describe how elements of the input JSON data should be mapped to RDF properties or classes. These hints, together with a representative subset of the input data, are sent to an external AI API to generate a candidate mapping. This prompt guides the language model to produce a valid RML mapping in Turtle syntax that conforms to the target JSON-LD structure. The prompt constrains the model to return a mapping definition adhering to RML rules, thereby improving reliability and syntactic correctness. In simplified form, the prompt contains: (i) role/instruction constraints, (ii) required prefixes and output format, (iii) representative input-data fragments, and (iv) user-provided mapping hints. Listing 2.1 shows a shortened excerpt of the prompt-construction logic.

The mapping editor is implemented using the CodeMirror component, a widely used web-based code editor that provides syntax highlighting and automatic indentation [Hav23]. Mapping validity is checked by the mapping service through debounced live validation. This allows users to inspect, refine, and validate the generated mapping directly within the interface before applying it to transform the JSON input into JSON-LD.

```
You are an assistant that generates RML mappings in Turtle syntax that convert JSON input into RDF.
Follow the RML specification strictly and avoid generating invalid RML constructs.
The output must contain ONLY valid Turtle mapping code and no explanations.

Logical source rules:
- Use rml:logicalSource
- Set rml:source to "Data.json"
- Use rml:referenceFormulation ql:JSONPath
- Use appropriate JSONPath iterators (e.g., "$.items[*]")

Object mapping / linking rules:
- In rr:objectMap, use exactly ONE of: rml:reference | rr:template | rr:constant
- If linking to another mapped resource, use rr:parentTriplesMap
- Do NOT combine rr:parentTriplesMap with rml:reference/rr:template/rr:constant
- For IRI objects from JSON values, use rr:template + rr:termType rr:IRI
- Do NOT use "$." inside rml:reference values

Prefix/output rules:
Use only provided prefixes (and user-provided ones), e.g.:
rr:  rml:  ql:  rdf:  xsd:
Declare prefixes at the top and return a single Turtle document.

Here is an example of the expected input and output
...

- Real input JSON subset
  {user input}

Additional user comments:
  {mapping hints}

- Final instruction: Return ONLY the RML mapping in valid Turtle syntax.
```

Listing 2.1: Part of the RML prompt template which guides the AI model

```
Convert my JSON Data to JSON-LD such that I have two nodes, each node representing a
    parameter or metric.
```

Listing 2.2: Sample RML hint which describes the mapping from JSON in 1.1 to a minimal JSON-LD in 2.4

```
@prefix rr: <http://www.w3.org/ns/r2rml#> .
@prefix rml: <http://semweb.mmlab.be/ns/rml#> .
@prefix ql: <http://semweb.mmlab.be/ns/ql#> .
@prefix ex: <http://example.com/ns#> .

<#ParameterMapping>
  rml:logicalSource [
    rml:source "Data.json" ;
    rml:referenceFormulation ql:JSONPath ;
    rml:iterator "$.parameters[*]"
  ] ;

  rr:subjectMap [
    rr:template "http://example.com/parameter/{name}" ;
    rr:class ex:Parameter
  ] ;

  rr:predicateObjectMap [
    rr:predicate ex:parameterName ;
    rr:objectMap [ rml:reference "name" ]
  ] ;
  rr:predicateObjectMap [
    rr:predicate ex:parameterValue ;
    rr:objectMap [ rml:reference "value" ]
  ] ;
  rr:predicateObjectMap [
    rr:predicate ex:parameterUnit ;
    rr:objectMap [ rml:reference "unit" ]
  ] .

<#MetricMapping>
  rml:logicalSource [
    rml:source "Data.json" ;
    rml:referenceFormulation ql:JSONPath ;
    rml:iterator "$.metrics[*]"
  ] ;

  rr:subjectMap [
    rr:template "http://example.com/metric/{name}" ;
    rr:class ex:Metric
  ] ;

  rr:predicateObjectMap [
    rr:predicate ex:metricName ;
    rr:objectMap [ rml:reference "name" ]
  ] ;
  rr:predicateObjectMap [
    rr:predicate ex:metricValue ;
    rr:objectMap [ rml:reference "value" ]
  ] ;
  rr:predicateObjectMap [
    rr:predicate ex:metricUnit ;
    rr:objectMap [ rml:reference "unit" ]
  ] .
```

Listing 2.3: RML mapping which converts JSON data in Listing 1.1 to JSON-LD in Listing 2.4

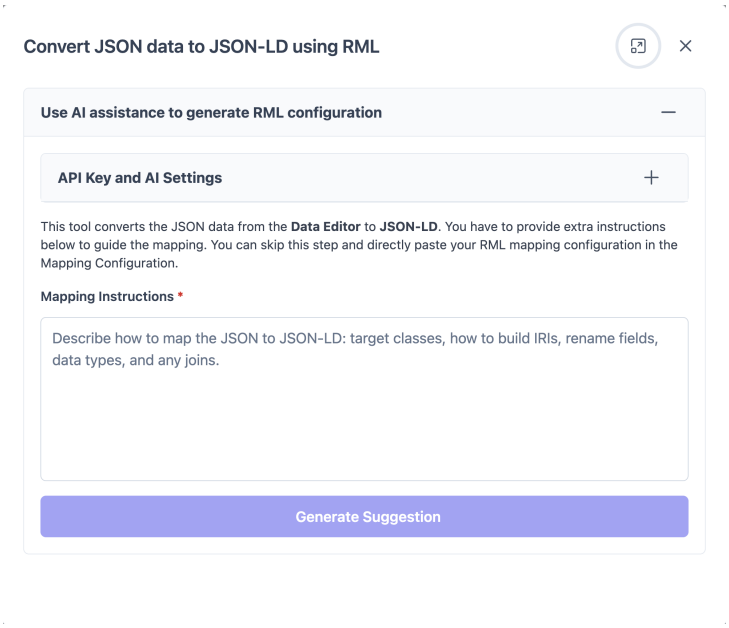


Figure 2.2: The RML mapping dialog for JSON-to-JSON-LD conversion.

Users may either provide a RML mapping in Turtle syntax directly or use the AI-assisted generation feature by supplying mapping hints. The resulting mapping is displayed in the CodeMirror editor, where it can be reviewed and edited as needed. Because AI-generated mappings may contain semantic or syntactic errors, users must verify them before execution. Any syntax errors introduced during mapping editing, as well as errors that occur after applying the mapping, are reported to the user to support debugging and correction. Once finalized, the mapping can be applied to convert the input JSON data into JSON-LD, which can then be further managed in the RDF authoring panel.

The examples in this section show that even a minimal hint can already produce a correct mapping for simple targets. In particular, the short natural-language hint shown above can still lead to a valid RML mapping (Listing 2.3) and a corresponding JSON-LD output (Listing 2.4).

However, realistic RML crafting is often more complex and usually requires precise guidance to obtain the intended semantic structure. Therefore, detailed hints are needed when the target output must follow specific vocabularies, resource patterns, and links between entities. Listing 2.5 shows a detailed example. With this level of instruction, the generated mapping matches the expected structure in Listing 1.7 and produce the intended JSON-LD in Listing 1.4.

After providing such hints, the configured AI model generates a mapping that is intended to follow the specified structure and semantics, although this is not always guaranteed in practice and

```json
{
  "@context": {
    "ex": "http://example.com/ns#"
  },
  "@graph": [
    {
      "@id": "http://example.com/metric/max_von_mises_stress",
      "@type": "ex:Metric",
      "ex:metricName": "max_von_mises_stress",
      "ex:metricUnit": "MPa",
      "ex:metricValue": {
        "@type": "http://www.w3.org/2001/XMLSchema#double",
        "@value": "2.997915075586333E8"
      }
    },
    {
      "@id": "http://example.com/parameter/element_size",
      "@type": "ex:Parameter",
      "ex:parameterName": "element_size",
      "ex:parameterUnit": "m",
      "ex:parameterValue": {
        "@type": "http://www.w3.org/2001/XMLSchema#double",
        "@value": "2.5E-2"
      }
    }
  ]
}
```

Listing 2.4: JSON-LD representation of the simulation result, obtained after applying the minimal RML mapping hint shown above

```
Create one main resource local:run_fenics_simulation of type m4i:Method.
This node links to parameter nodes using m4i:hasParameter and to metric nodes using m4i:investigates.

Iterate over the arrays parameters[*] and metrics[*] separately to create parameter and metric nodes.

For each entry in these arrays:

Create a resource with IRI local:variable_{name} using the "name" field.
Set the type to schema:PropertyValue.
Set rdfs:label to the "name" value.
Set schema:value to the "value".
Map the "unit" field to schema:unitCode as an IRI from the QUDT unit vocabulary using the template
      http://qudt.org/vocab/unit/{unit}.

Connect the method node to parameter nodes using m4i:hasParameter, and to metric nodes using m4i:investigates.

Use these prefixes in the mapping:
m4i: http://w3id.org/nfdi4ing/metadata4ing#
schema: https://schema.org/
rdfs: http://www.w3.org/2000/01/rdf-schema#
unit: http://qudt.org/vocab/unit/
local: https://www.example.org/
```

Listing 2.5: Sample RML hints which describe the mapping from JSON in 1.1 to JSON-LD in 1.4

largely depends on the capability of the underlying model. Once the user reviews and confirms the mapping, it can be executed to transform the input JSON data into a semantically enriched JSON-LD representation, as illustrated in Figure 2.3.

2.1.2 Panel Composition and UI Structure

The core editing surface is implemented in `RdfEditorPanel.vue` as a PrimeVue Tabs[3] container. The implementation is primarily organized into two tabs:

- **Context tab**: for editing `@context` definitions.
- **Triples tab**: for browsing and editing RDF triples.

This explicit split mirrors the conceptual separation in JSON-LD between vocabulary declarations and graph assertions [SKL14], and therefore supports both semantic modeling tasks (namespace/prefix handling) and graph authoring tasks (subject-predicate-object manipulation) in one workflow.

When the current document is invalid or not in the expected JSON-LD form, both tabs become visually disabled. This prevents accidental modifications when parsing preconditions are not satisfied.

2.1.3 Context Tab

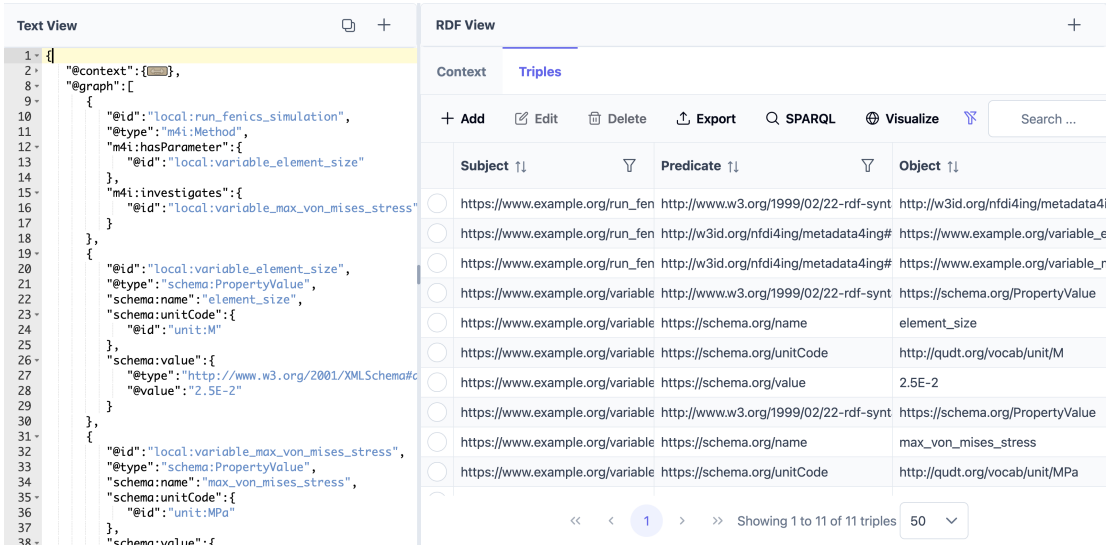
The Context tab is implemented by `RdfContextEditorPanel.vue`. It reuses MetaConfigurator's GUI editor and applies a predefined JSON Schema so that `@context` can be edited through structured forms rather than plain text only. The selection updates are synchronized with the text editor view, enabling users to navigate to and manipulate context entries consistently with other panels.

2.1.4 Triples Tab

The Triples tab is implemented by `RdfTripleEditorPanel.vue`. It shows triples in a PrimeVue `DataTable` [4] with:

- paging, sorting, and column/global filtering,
- single-row selection synchronized with document path selection in the text editor,
- actions for add, edit, delete, export, SPARQL, and visualization.

[3] `https://primevue.org/tabs/`
[4] `https://primevue.org/datatable/`



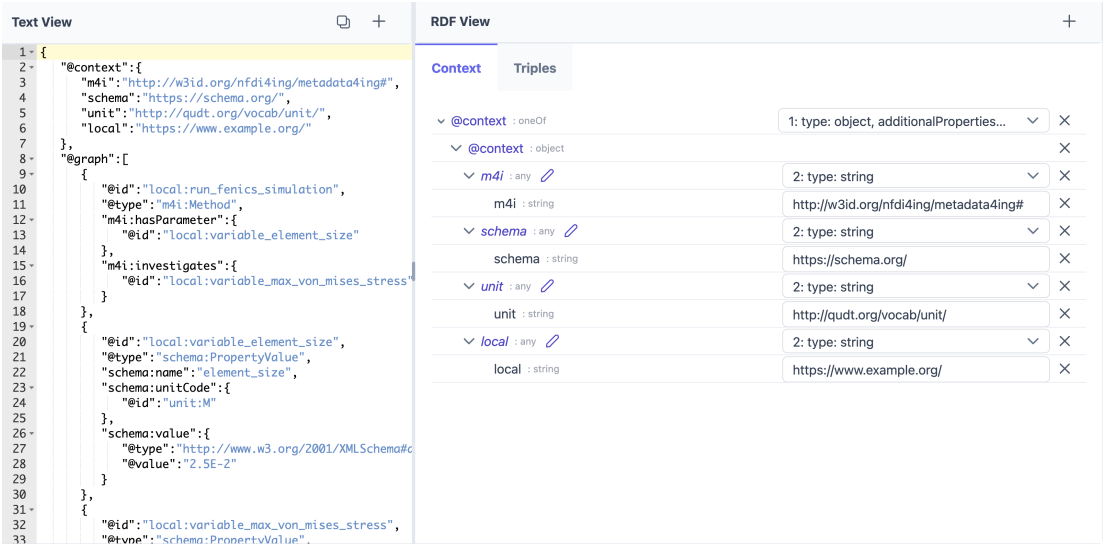


Figure 2.3: JSON-LD output obtained by applying the mapping from Listing 1.7, focused on the Triples and Context tabs, respectively.

Triple editing is handled through `TripleDetailsDialog.vue`, which allows controlled entry of subject, predicate, and object terms, including typed literals with XML Schema datatypes. Changes are executed through `TripleEditorService` and propagated to the central RDF store. Figure 2.4 shows the edit modal for triple editing.

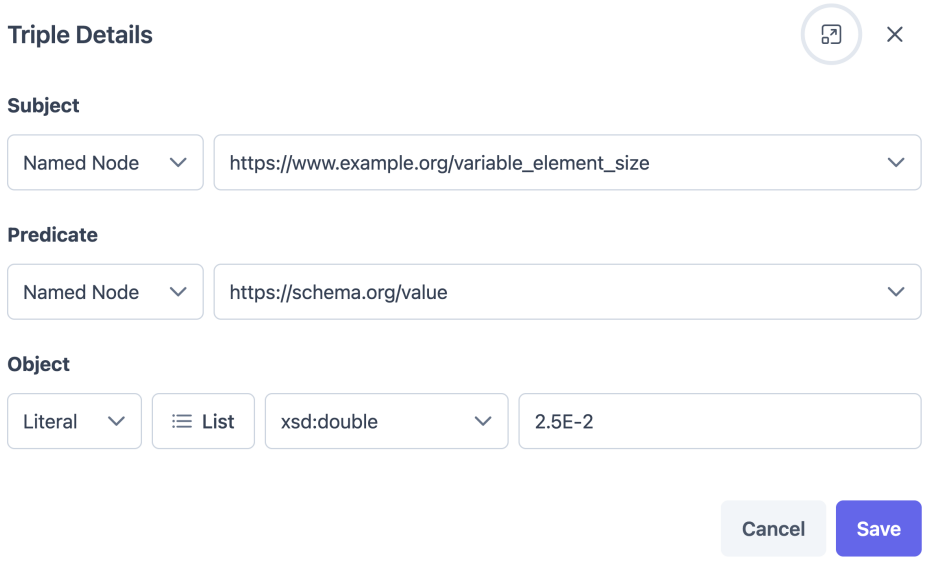


Figure 2.4: Edit modal for triple editing.

Data changes originate from two primary sources: the Text View (raw JSON-LD data) and the RDF Panel. In this simplified description, updates appear to flow directly between these components. In reality, however, all data is managed through a central store (`dataSource`) that acts as the single source of truth, while the individual panels remain decoupled and do not communicate with each other directly. At this layer, `managedData.ts` provides synchronized access to both parsed data and its text representation, supports path-based updates, and preserves temporarily unparseable text input without discarding the last valid data state. For example, when a change is made in the Text View, `rdfStoreManager.ts` reconstructs the RDF graph within this central store, which in turn triggers an update in the RDF Panel (Figure 2.5).

Conversely, when a triple is edited, removed, or added in Triples tab, `TripleEditorService` validates and applies the change through `rdfStoreManager.ts`. If successful, `jsonLdNodeManager.ts` rebuilds JSON-LD from the updated store and writes it back to the `dataSource`, which updates the Text View accordingly (Figure 2.6). Changes in the Context tab is propagated in the same way as changes to the GUI View, since underlying data is still treated as JSON data and processed by the same synchronization workflow.

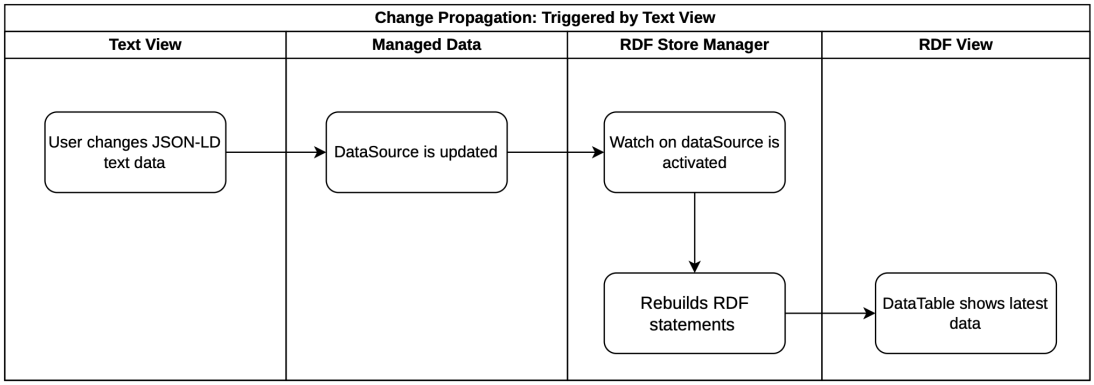


Figure 2.5: Synchronization process illustrating how modifications made in the Text View are propagated and reflected in the RDF View.

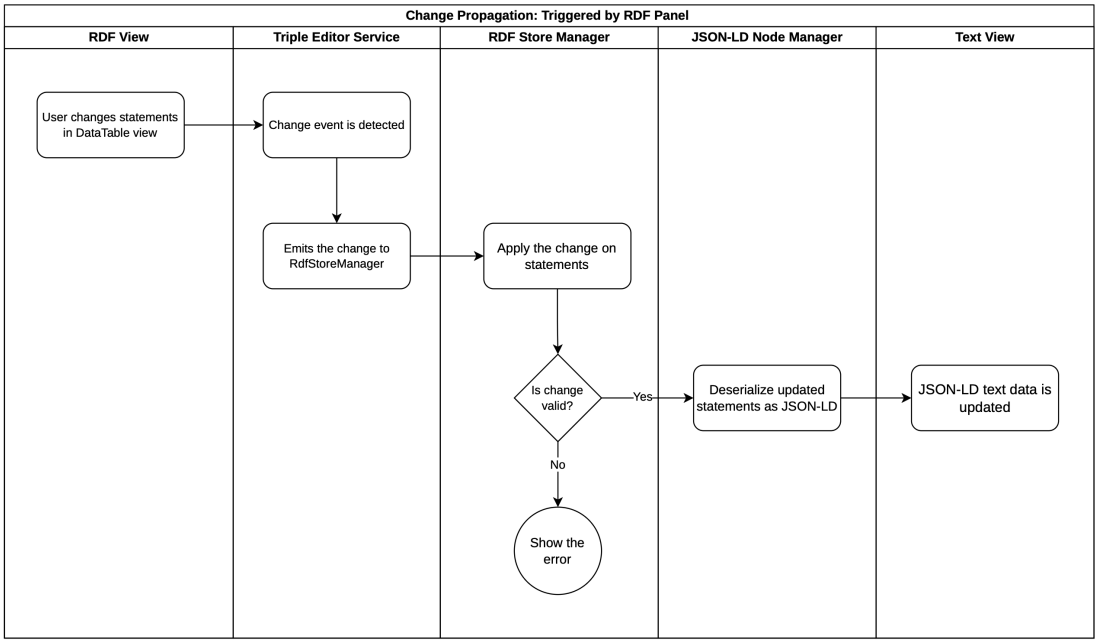


Figure 2.6: Workflow showing how changes originating in the RDF View are transferred back and represented in the Text View.

2.1.5 RDF Store and Data Synchronization

The internal semantic state is managed by `rdfStoreManager.ts`, which uses `rdflib.js` to parse and serialize data [lin26]. Because the synchronization logic operates on statement-level objects throughout this subsection, we first define the term *RDFLib Statement* used in the following description.

Definition 2.1 (RDFLib Statement)
In this implementation, an RDFLib Statement is the runtime representation of one RDF triple of the form (s, p, o), where s (subject), p (predicate), and o (object) are RDF terms. In `rdflib.js`*, these terms are represented as* `NamedNode`*,* `BlankNode`*, and* `Literal` *objects, and each triple is stored as a Statement. Briefly, a* `NamedNode` *represents an identified resource via an IRI (e.g., ontology classes, predicates, or linked entities), a* `BlankNode` *represents an unnamed local resource without a global IRI, and a* `Literal` *represents a concrete value such as text, number, or date (optionally with datatype or language tag). A collection of such Statements forms the in-memory graph used by the panel [lin26].*

The `rdflib.js` library was selected because it provides mature browser-side handling of JSON-LD/RDF terms and integrates directly with statement-level editing operations used by the panel. On each source-data update, the manager:

1. resolves `@context` data into namespace mappings,
2. parses JSON-LD into an indexed RDF graph and derives a reactive list of Statements,
3. exposes reactive Statements, warnings, and errors to UI components.

This definition is directly reflected in the implementation workflow: the internal store parses JSON-LD into a reactive list of Statement objects, the triples table renders this list, and editing actions create or modify individual statements for add/edit/delete operations. Statement-level matching (subject, predicate, object) is used to synchronize table selection with text-editor paths.

To keep the JSON editor and RDF authoring panel consistent, the implementation maps RDF statements back into JSON-LD data and computes path correspondences between selected triples and document positions. This enables bidirectional synchronization: selecting a triple can focus the matching JSON path, and raw JSON data changes trigger store rebuilds.

2.1.6 Linked Selection Between Text View and RDF View

Linked selection between the Text View and the RDF View enables coordinated highlighting of corresponding elements across both representations. When a user selects a section of the JSON-LD document, the corresponding RDF statement is identified and highlighted in the triple table, and vice versa. This maintains a consistent visual selection state across both views. Figure 2.7 illustrates this behavior. Algorithms 2.1 and 2.2 describe the implementation of linked selection in both directions.

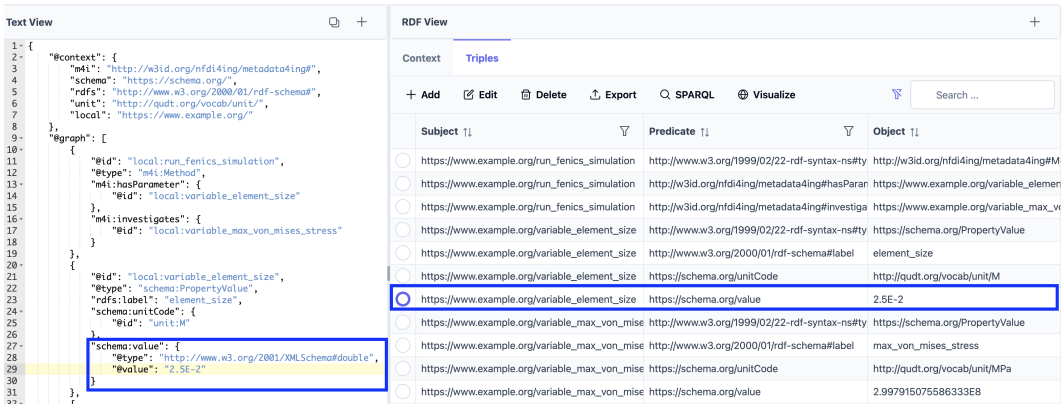


Figure 2.7: Linked selection between Text View and RDF View. The highlighted JSON fragment corresponds to the selected triple in the triple table, and vice versa.

Algorithmus 2.1 Linked selection from Text View to RDF View

1: **Input:** current JSON path selection, full JSON-LD document, current triple table rows
2: Extract subject (`@id`), predicate, and object at the selected path in the JSON-LD document
3: Build a minimal JSON-LD fragment containing:
   a) original `@context`
   b) one `@graph` node with extracted subject, predicate, and object
4: Parse the minimal fragment with RDFLib into a temporary graph
5: **if** the temporary graph contains exactly one statement **then**
6: Compare that statement against each triple table statement by subject, predicate, and object equality
7: **if** only one matching statement is found **then**
8: Determine its row index in the triple table
9: Set triple table selection to the matching row
10: Scroll/focus the table to the selected row

Algorithmus 2.2 Linked selection from RDF View to Text View

Input: selected triple table statement (s, p, o), current JSON-LD document
Read subject s, predicate p, and object o from the selected table row
Find candidate node in `@graph` whose `@id` matches s
if no candidate node is found **then**
    return without navigation
Find predicate entry in the candidate node that matches p
    (consider both prefixed terms and full IRIs as equivalent)
if no matching predicate entry is found **then**
    return without navigation
Match object o within the predicate value
    (as `@id`, `@value`, or scalar value)
if an object match is found **then**
    Compute the corresponding JSON path
    Move editor cursor/focus to that JSON path

2.2 Query with SPARQL

The SPARQL query functionality is implemented in `SparqlEditor.vue` as a dialog with three tabs: **Query View**, **Result View**, and **Visualization**. During implementation, RDFLib query capabilities were evaluated first, but they proved limited for the required in-browser query workflow and did not provide full support for current SPARQL 1.2 draft specifications. Query execution is therefore handled in-browser via Comunica's `QueryEngine` [Ass26a]. According to Comunica's official documentation, the engine supports SPARQL 1.2 draft query-related specifications [Ass26b].

In the **Query View**, users can write and validate queries in a CodeMirror editor [Hav23] and execute them against the current in-memory RDF graph. Query syntax is checked through debounced live validation using the SparqlJS parser, which provides immediate feedback while editing [Vc26]. The same view also provides AI-assisted SPARQL generation. Similar to the AI-assisted RML workflow described in Section 2.1.1, user hints are combined with prompt-engineering instructions and a representative subset of the current JSON-LD data, then sent to the configured AI-endpoint to generate a draft SPARQL query. In simplified form, the prompt includes: (i) output constraints (return only SPARQL), (ii) relevant prefixes/context, (iii) representative graph fragments, and (iv) the user hint. Figure 2.8 shows the dialog with an AI-assisted query suggestion in the Query View.

To demonstrate the AI-assisted workflow, we assume that the simulation JSON from Listing 1.1 has been extended with additional `element_size` and `max_von_mises_stress` value pairs (e.g., from multiple simulation runs), and then transformed into JSON-LD using the mapping workflow

described earlier. In this extended dataset, metric measurements are available for multiple element sizes, for example `0.003125`, `0.00625`, `0.0125`, `0.05` and `0.1`.

In this scenario, the user provides a natural-language hint in the Query View to request a query that returns paired values of `element_size` and `max_von_mises_stress`. An example hint is shown in Listing 2.6. The hint and a subset of the current JSON-LD graph are sent to the AI API, which proposes a draft query that may still contain syntactic or semantic errors. Therefore, the generated query must be reviewed and, if necessary, refined by the user before execution. A representative generated query is shown in Listing 2.7.

```
Return pairs of element_size and max_von_mises_stress.
For each run, show the numeric element size value and the numeric stress value.
Sort by element_size ascending.
```

Listing 2.6: Example user hint for AI-assisted SPARQL generation

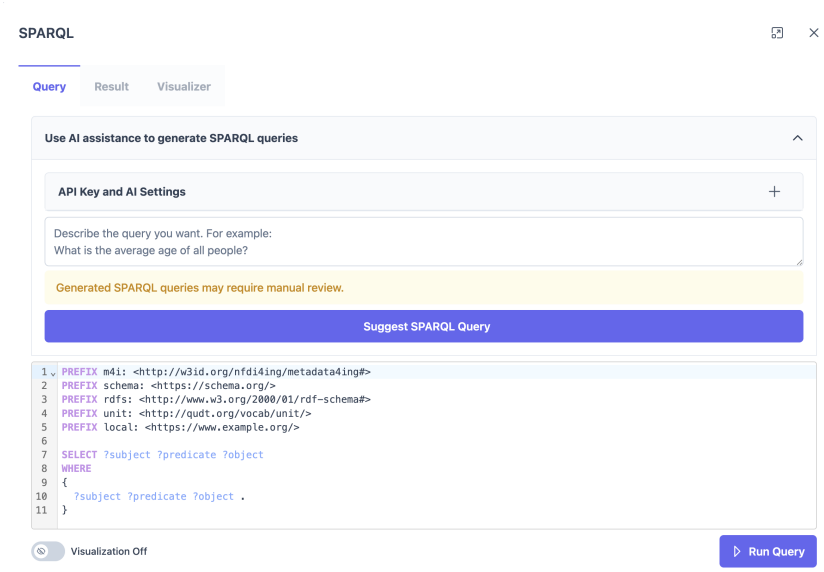


Figure 2.8: SPARQL query dialog, with AI-assisted query generation.

This showcase illustrates the intended interaction pattern: AI is used to accelerate query authoring, while final semantic and syntactic validation remains under user control before execution. Human-in-the-loop review is essential in both AI-generated RML mappings and AI-generated SPARQL queries, and users can inspect and refine both results before applying or executing them.

The dialog supports two execution modes controlled by a toggle: `Visualization Off` and `Visualization On`. With `Visualization Off`, standard SELECT-style querying is used and results are shown in

```
PREFIX m4i: <http://w3id.org/nfdi4ing/metadata4ing#>
PREFIX schema: <https://schema.org/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX unit: <http://qudt.org/vocab/unit/>
PREFIX local: <https://www.example.org/>

SELECT ?element_size ?max_von_mises_stress WHERE {
  ?method a m4i:Method .
  ?method m4i:hasParameter ?element_size_param .
  ?element_size_param a schema:PropertyValue ;
                      schema:value ?element_size .
  ?method m4i:investigates ?stress_param .
  ?stress_param a schema:PropertyValue ;
                schema:value ?max_von_mises_stress .
} ORDER BY ASC(?element_size)
```

Listing 2.7: AI-suggested SPARQL query for `element_size` vs. `max_von_mises_stress` pairs

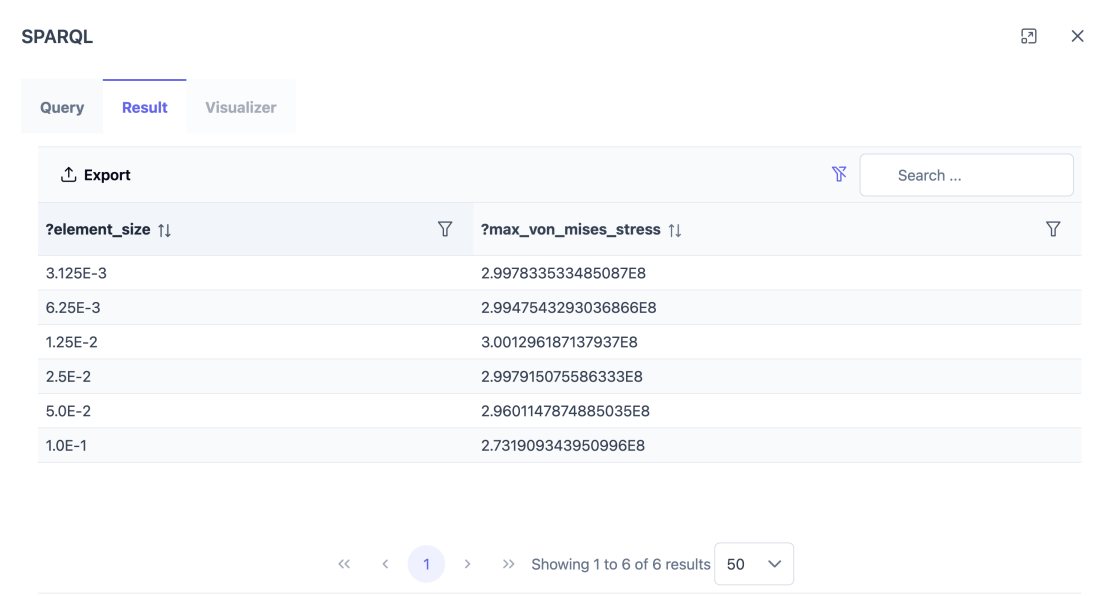


Figure 2.9: Result after running SPARQL query in Figure 2.8.

the **Result View**. There, the PrimeVue DataTable columns are generated dynamically from the returned variables, and export is provided as CSV. Figure 2.10 shows a plot which is generated from the CSV export of the query result, visualizing the relationship between element size and maximum von Mises stress.

With `Visualization On`, the query mode switches to CONSTRUCT-based graph output; the same result table is still populated, but export options target RDF serializations (e.g., Turtle, N-Triples, RDF/XML).

The **Visualization** tab renders the query result graph in read-only mode. Its interaction model and rendering details are discussed in the Section 2.3.

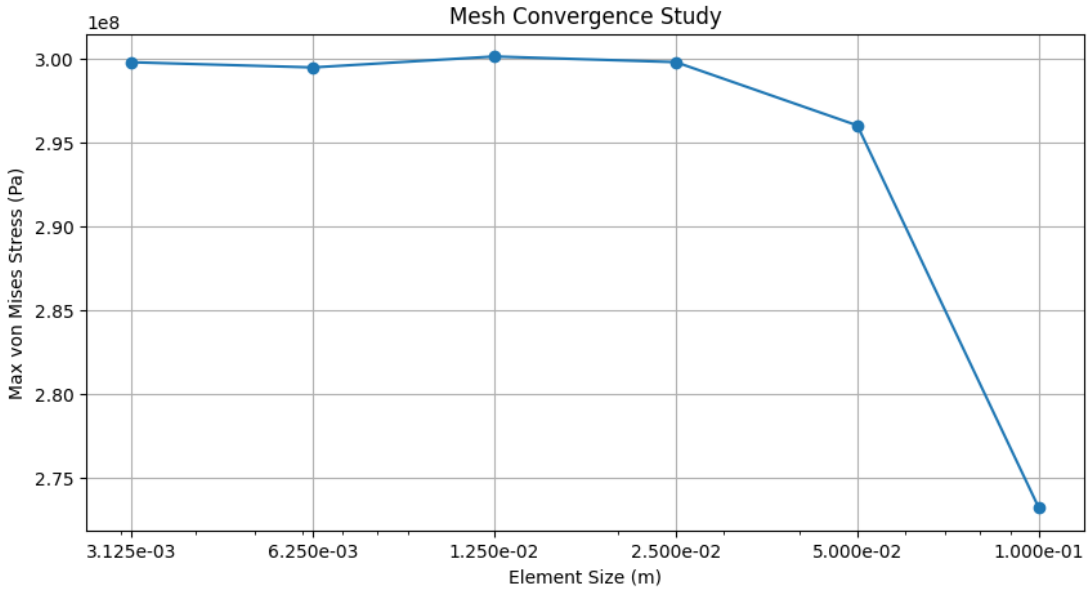


Figure 2.10: CSV export visualized: `element_size` vs. `max_von_mises_stress` trends.

2.3 KG Visualization

KG rendering is implemented in `RdfVisualizer.vue` using Cytoscape.js [FLH+16], with the COSE-Bilkent layout extension for force-directed graph placement. The visualization presents resources as nodes and semantic relations as directed edges, while preserving readable labels through namespace-aware prefix shortening.

The view provides interactive graph navigation features including zoom in/out, fit-to-view, and zoom-to-selected-node actions. For performance and usability, large graphs are detected before rendering and users are prompted to continue or cancel. In addition, graph export is supported as an image, and an optional animation mode can recompute the layout to improve readability after edits.

Node inspection and editing support are integrated through `RdfVisualizerPropertiesView.vue`. When a node is selected, a side panel shows the node identifier and its properties, including linkable IRIs. The same panel exposes lightweight authoring actions (add/edit/delete property, rename/delete node, add node), so users can inspect and refine graph content directly from the visualization workflow.

The visualization tab can be opened in two ways. First, it can be opened directly from the Triples tab of the RDF panel. In this mode, the visualization is editable, and user interactions are propagated back to the main JSON-LD data and RDF store. Second, it can be opened from the SPARQL dialog when the `Visualization On` toggle is enabled. In that case, the graph is built from the user's SPARQL CONSTRUCT result and is shown in read-only mode, because returned triples are not guaranteed to map one-to-one to statements in the internal RDF store.

For editable visualization workflows, undo and redo actions are also integrated in the graph toolbar. This allows users to safely revise node/property edits while keeping visualization and underlying data synchronized.

To improve navigation in larger graphs, the visualization also provides node search via PrimeVue AutoComplete[5] component: users receive suggestions of available nodes while typing, can quickly select the intended node, and the view automatically focuses on it.

The visualization also supports clickable hyperlinks for semantic terms. Predicate labels can open their ontology definitions; for example, `m4i:hasParameter` links to predicate definition in the ontology[6]. For object values, `NamedNode` are handled context-sensitively: if the referenced node exists in the current graph, clicking it focuses that node in the visualization; if it points to an external resource, clicking opens the external definition (e.g., `unit:M` linking to `https://qudt.org/vocab/unit/M`).

[5] `https://primevue.org/autocomplete/`

[6] `https://nfdi4ing.pages.rwth-aachen.de/metadata4ing/metadata4ing/index.html#hasParameter`

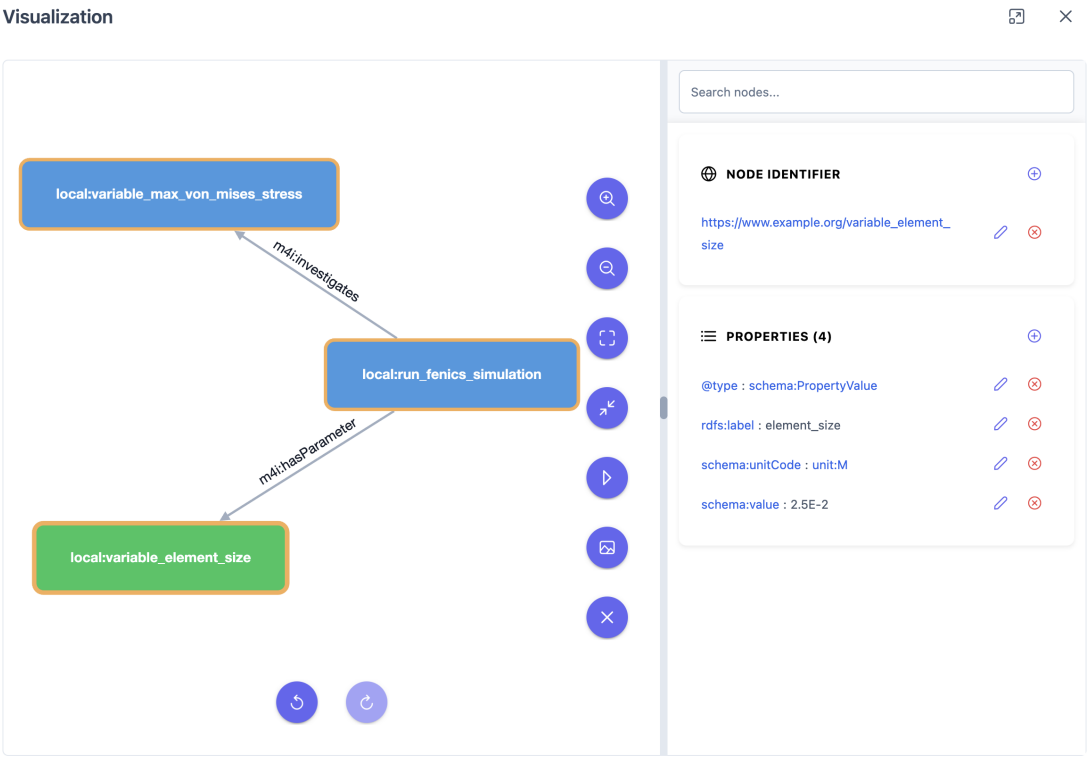


Figure 2.11: KG visualization of the JSON-LD data from Listing 1.4, rendered in the Visualization dialog.

3 Application in Chemistry

This chapter presents the end-to-end application of the thesis workflow to MOF synthesis data. Structured experiment records are first modeled in JSON, then transformed into semantically enriched RDF/JSON-LD using RML, and finally prepared for ontology-aware querying and graph-based analysis.

In line with the motivation in Chapters 1 and 2, the main idea is to preserve laboratory protocol detail (materials, actions, and conditions) while adding formal semantics so that the same data can be interpreted consistently across tools and datasets [NEB+26].

3.1 JSON Structure of the Chemical Dataset

MOF synthesis refers to the stepwise preparation of crystalline coordination materials from metal precursors and organic linkers under controlled laboratory conditions. Beyond the final product, the process is characterized by full protocol context, including reagents, roles, action sequence, and process conditions, which should be represented in structured, machine-readable form [NEB+26].

Listings 3.1 and 3.2 show a representative subset of this dataset. Each top-level entry under `Synthesis[*]` encodes one complete run (e.g., `S-1`, `S-2`) with four core components: hardware setup, run metadata, ordered procedure phases (Prep/Reaction/Workup), and reagent inventory with functional roles.

3.2 CHMO Ontology

The Chemical Methods Ontology (CHMO) provides a controlled vocabulary for chemical methods and related experimental concepts, including data-acquisition techniques, preparation and separation procedures, and synthesis methods [RO26]. Within the broader OBO ecosystem, CHMO is designed to complement other ontologies such as the Ontology for Biomedical Investigations (OBI), which captures more general investigation processes and experimental context [BBB+16; JMO+21].

```
{
  "Synthesis": [
    {
      "Hardware": { "Component": [{ "_id": "S-1", "_type": "glass vial" }] },
      "Metadata": { "_description": "S-1", "_product": "MIL-88B (Fe)" },
      "Procedure": {
        "Prep": {
          "Step": [
            { "_vessel": "S-1", "$xml_type": "Add", "_amount": { "Unit": "millimole", "Value": 0.4},
                "_reagent": "FeCl3", "_order": 0 },
            { "_vessel": "S-1", "$xml_type": "Add", "_amount": { "Unit": "millimole", "Value": 0.4},
                "_reagent": "Benzene-1,4-dicarboxylic acid", "_order": 1 },
            { "_vessel": "S-1", "$xml_type": "Add", "_amount": { "Unit": "millilitre", "Value": 4}, "_reagent":
                "DMF", "_order": 2 }
          ]
        },
        "Reaction": {
          "Step": [
            { "_vessel": "S-1", "$xml_type": "Sonicate", "_time": { "Value": 30.0, "Unit": "minute"}, "_order":
                0 },
            { "_comment": "In: oven", "_vessel": "S-1", "$xml_type": "HeatChill", "_temp": { "Unit": "celsius",
                "Value": 120.0}, "_time": { "Value": 1, "Unit": "day"}, "_order": 1 }
          ]
        },
        "Workup": {
          "Step": [
            { "_vessel": "S-1", "$xml_type": "WashSolid", "_solvent": "EtOH", "_order": 0 },
            { "_vessel": "S-1", "$xml_type": "Dry", "_temp": { "Unit": "celsius", "Value": 120.0}, "_time": {
                "Value": 1, "Unit": "day"}, "_pressure": { "Unit": "pascal", "Value": 0}, "_order": 1 }
          ]
        }
      },
      "Reagents": {
        "Reagent": [
          { "_id": "FeCl3", "_name": "FeCl3", "_role": "substrate" },
          { "_id": "Benzene-1%252C4-dicarboxylic%2520acid", "_name": "Benzene-1,4-dicarboxylic acid", "_role":
              "ligand" },
          { "_id": "DMF", "_name": "DMF", "_role": "solvent" },
          { "_id": "EtOH", "_name": "EtOH", "_role": "solvent" }
        ]
      }
    }
  ]
}
```


Listing 3.1: Representative JSON excerpt from the MOF synthesis dataset for `S-1`

```
{
  "Synthesis": [
    {
      "Hardware": { "Component": [{ "_id": "S-2", "_type": "glass vial" }] },
      "Metadata": { "_description": "S-2", "_product": "MIL-88B+MIL101 (mix phases)" },
      "Procedure": {
        "Prep": {
          "Step": [
            { "_vessel": "S-2", "$xml_type": "Add", "_amount": { "Unit": "millimole", "Value": 0.4},
                 "_reagent": "FeCl3\\u00B76H2O", "_order": 0 },
            { "_vessel": "S-2", "$xml_type": "Add", "_amount": { "Unit": "millimole", "Value": 0.4},
                 "_reagent": "Benzene-1,4-dicarboxylic acid", "_order": 1 },
            { "_vessel": "S-2", "$xml_type": "Add", "_amount": { "Unit": "millilitre", "Value": 4}, "_reagent":
                 "DMF", "_order": 2 }
          ]
        },
        "Reaction": {
          "Step": [
            { "_vessel": "S-2", "$xml_type": "Sonicate", "_time": { "Value": 30.0, "Unit": "minute"}, "_order":
                 0 },
            { "_comment": "In: oven", "_vessel": "S-2", "$xml_type": "HeatChill", "_temp": { "Unit": "celsius",
                 "Value": 120.0}, "_time": { "Value": 1, "Unit": "day"}, "_order": 1 }
          ]
        },
        "Workup": {
          "Step": [
            { "_vessel": "S-2", "$xml_type": "WashSolid", "_solvent": "EtOH", "_order": 0 },
            { "_vessel": "S-2", "$xml_type": "Dry", "_temp": { "Unit": "celsius", "Value": 120.0}, "_time": {
                 "Value": 1, "Unit": "day"}, "_pressure": { "Unit": "pascal", "Value": 0}, "_order": 1 }
          ]
        }
      },
      "Reagents": {
        "Reagent": [
          { "_id": "FeCl3%25C2%25B76H2O", "_name": "FeCl3\\u00B76H2O", "_role": "substrate" },
          { "_id": "Benzene-1%252C4-dicarboxylic%2520acid", "_name": "Benzene-1,4-dicarboxylic acid", "_role":
               "ligand" },
          { "_id": "DMF", "_name": "DMF", "_role": "solvent" },
          { "_id": "EtOH", "_name": "EtOH", "_role": "solvent" }
        ]
      }
    }
  ]
}
```


Listing 3.2: Representative JSON excerpt from the MOF synthesis dataset for `S-2`

Using CHMO serves two purposes. First, it anchors synthesis resources to a domain-specific ontology rather than leaving them as generic data records. Second, it improves interoperability, because CHMO-typed entities can be combined with OBI- and Chemical Entities of Biological Interest (ChEBI)-based descriptions in a shared RDF graph.

3.3 RML Mapping for CHMO Integration

The RML mappings in Listings 3.3 and 3.4 show how the JSON structures defined in Listings 3.1 and 3.2 are transformed into typed RDF resources, as described in Section 1.2.7. In MetaConfigurator, this is the operational step that bridges conventional JSON input and the RDF/JSON-LD authoring and query workflow.

Listings 3.3 and 3.4 demonstrate how the chemistry-oriented ontology design is operationalized as executable RML. The first listing defines the synthesis-level mapping context: `rml:iterator "$.Synthesis[*]"` selects each run in the input data, and `rr:subjectMap` creates one stable resource per run. By assigning `rr:class chmo:0000410`, each generated subject is explicitly typed as a CHMO-grounded synthesis entity rather than a generic node.

The same mapping block also shows how experimental metadata is semantically attached to the synthesis resource. Fields such as vessel identifier, vessel type, human-readable run label, and product are transferred through `rr:predicateObjectMap`. This mapping pattern preserves the operational laboratory context while simultaneously lifting the data into an ontology-aware representation that can be interpreted by semantic tools.

The second listing extends this principle to reagents and preparation steps. Reagent resources are typed with `obo:CHEBI_24431`, aligning them with ChEBI-based chemical entities, while preparation-step resources are typed with `obo:OBI_0000070`, aligning procedural actions with OBI-based process semantics. As a result, the mapping does not merely copy JSON values into triples; it establishes a typed graph structure in which substances, methods, and process phases can be distinguished and queried systematically.

From a workflow perspective, this is crucial for the integration goals discussed in Chapters 1 and 2. Once the mapped graph is loaded in MetaConfigurator, users can formulate SPARQL queries over synthesis-level entities, reagent roles, and process steps without relying on fragile path-specific JSON logic. The same typed structure also improves graph visualization readability, because nodes and edges represent explicit domain semantics rather than implicit document structure.

A central semantic effect of the mapping is ontology-based typing in `rr:class`. These class assignments transform plain JSON objects into explicit domain entities. For example, the mappings in Listings 3.3 and 3.4 assign the following classes:

```
@prefix rr: <http://www.w3.org/ns/r2rml#> .
@prefix rml: <http://semweb.mmlab.be/ns/rml#> .
@prefix ql: <http://semweb.mmlab.be/ns/ql#> .
@prefix ex: <http://example.org/> .
@prefix chmo: <http://purl.obolibrary.org/obo/CHMO_> .
@prefix obo: <http://purl.obolibrary.org/obo/> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

<#SynthesisSource> rml:source "Data.json" ; rml:referenceFormulation ql:JSONPath ; rml:iterator "
     $.Synthesis[*]" .

<#SynthesisTriplesMap>
  rml:logicalSource <#SynthesisSource> ;
  rr:subjectMap [ rr:template "http://example.org/synthesis/{Hardware.Component[0]._id}" ; rr:class
       chmo:0000410 ] ;
  rr:predicateObjectMap [ rr:predicate ex:componentID ; rr:objectMap [ rml:reference "
       Hardware.Component[0]._id" ] ] ;
  rr:predicateObjectMap [ rr:predicate ex:componentType ; rr:objectMap [ rml:reference "
       Hardware.Component[0]._type" ] ] ;
  rr:predicateObjectMap [ rr:predicate rdfs:label ; rr:objectMap [ rml:reference "Metadata._description" ] ] ;
  rr:predicateObjectMap [ rr:predicate ex:hasProduct ; rr:objectMap [ rml:reference "Metadata._product" ] ] .
```

Listing 3.3: Part of RML mapping focused on synthesis-level hardware and metadata

```
<#ReagentSource> rml:source "Data.json" ; rml:referenceFormulation ql:JSONPath ; rml:iterator "
     $.Synthesis[*].Reagents.Reagent[*]" .

<#ReagentTriplesMap>
  rml:logicalSource <#ReagentSource> ;
  rr:subjectMap [ rr:template "http://example.org/reagent/{_id}" ; rr:class obo:CHEBI_24431 ] ;
  rr:predicateObjectMap [ rr:predicate rdfs:label ; rr:objectMap [ rml:reference "_name" ] ] ;
  rr:predicateObjectMap [ rr:predicate ex:role ; rr:objectMap [ rml:reference "_role" ] ] .

<#PrepStepSource> rml:source "Data.json" ; rml:referenceFormulation ql:JSONPath ; rml:iterator "
     $.Synthesis[*].Procedure.Prep.Step[*]" .

<#PrepStepTriplesMap>
  rml:logicalSource <#PrepStepSource> ;
  rr:subjectMap [ rr:template "http://example.org/step/prep/{_vessel}-{_order}" ; rr:class obo:OBI_0000070 ] ;
  rr:predicateObjectMap [ rr:predicate ex:phase ; rr:objectMap [ rr:constant "Prep" ] ] ;
  rr:predicateObjectMap [ rr:predicate ex:action ; rr:objectMap [ rml:reference "$xml_type" ] ] ;
  rr:predicateObjectMap [ rr:predicate ex:amountValue ; rr:objectMap [ rml:reference "_amount.Value" ] ] ;
  rr:predicateObjectMap [ rr:predicate ex:amountUnit ; rr:objectMap [ rml:reference "_amount.Unit" ] ] .
```

Listing 3.4: Part of RML mapping focused on reagents and preparation steps

Table 3.1: Ontology classes used in `rr:class` assignments in Listings 3.3 and 3.4

Class (Prefix Form)	Ontology	Definition in this mapping context
`chmo:0000410`	CHMO	Types each synthesis run as a chemistry synthesis entity.
`obo:CHEBI_24431`	ChEBI	Types each reagent resource as a chemical entity (e.g., ligand, solvent, substrate).
`obo:OBI_0000070`	OBI	Types each procedure-step resource as an experimental/process step entity.

This is important because JSON keys such as `Synthesis`, `Reagent`, and `Step` are structural labels only. After RML transformation, they become ontology-grounded RDF nodes that can be queried by meaning and reused across datasets. Listing 3.5 shows part of the transformed data in JSON-LD format, representing synthesis run for `S-1`.

3.4 AI-Assisted SPARQL Generation

After applying the full mapping in Listing 3.3 and Listing 3.4, users can query the resulting RDF graph directly with SPARQL. In practical laboratory workflows, however, writing correct SPARQL often becomes a bottleneck, especially for users with strong domain expertise but limited query-language experience. Therefore, this chapter applies the same AI-assisted interaction pattern introduced in Chapter 2: users formulate an intent in natural language, and a configured Large Language Model (LLM) proposes a first query draft.

For the MOF dataset, the main benefit is that users can start from experimental questions (e.g., "Which Prep steps used which reagent, and in what amount?") instead of manually constructing triple patterns from scratch. Listing 3.6 shows such a concise user hint. Based on this hint and a representative subset of the current JSON-LD/RDF graph, the LLM generates a candidate query, shown in Listing 3.7. Figure 3.1 shows the result of executing this query, which retrieves all preparation steps with their associated reagents and amounts for each synthesis run.

This workflow lowers the entry barrier for SPARQL authoring, reduces trial-and-error iterations in the query editor, and helps domain experts focus on scientific interpretation rather than query syntax details. At the same time, the generated query remains a draft artifact: it can contain modeling assumptions or predicate choices that require correction. Consequently, a user-controlled review step remains mandatory before execution.

```
{
  "@context": {
    "ex": "http://example.org/",
    "chmo": "http://purl.obolibrary.org/obo/CHMO_",
    "obo": "http://purl.obolibrary.org/obo/",
    "rdfs": "http://www.w3.org/2000/01/rdf-schema#",
    "xsd": "http://www.w3.org/2001/XMLSchema#"
  },
  "@graph": [
    { "@id": "ex:reagent/Benzene-1%25252C4-dicarboxylic%252520acid", "@type": "obo:CHEBI_24431", "ex:role":
        "ligand", "rdfs:label": "Benzene-1,4-dicarboxylic acid" },
    { "@id": "ex:reagent/DMF", "@type": "obo:CHEBI_24431", "ex:role": "solvent", "rdfs:label": "DMF" },
    { "@id": "ex:reagent/EtOH", "@type": "obo:CHEBI_24431", "ex:role": "solvent", "rdfs:label": "EtOH" },
    { "@id": "ex:reagent/FeCl3", "@type": "obo:CHEBI_24431", "ex:role": "substrate", "rdfs:label": "FeCl3" },

    { "@id": "ex:step/prep/S-1-0", "@type": "obo:OBI_0000070", "ex:action": "Add", "ex:amountUnit":
        "millimole", "ex:amountValue": { "@type": "xsd:double", "@value": "4.0E-1" }, "ex:phase": "Prep",
        "obo:RO_0000057": { "@id": "ex:reagent/FeCl3" } },
    { "@id": "ex:step/prep/S-1-1", "@type": "obo:OBI_0000070", "ex:action": "Add", "ex:amountUnit":
        "millimole", "ex:amountValue": { "@type": "xsd:double", "@value": "4.0E-1" }, "ex:phase": "Prep",
        "obo:RO_0000057": { "@id": "ex:reagent/Benzene-1%25252C4-dicarboxylic%252520acid" } },
    { "@id": "ex:step/prep/S-1-2", "@type": "obo:OBI_0000070", "ex:action": "Add", "ex:amountUnit":
        "millilitre", "ex:amountValue": { "@type": "xsd:integer", "@value": "4" }, "ex:phase": "Prep",
        "obo:RO_0000057": { "@id": "ex:reagent/DMF" } },

    { "@id": "ex:step/reaction/S-1-0", "@type": "obo:OBI_0000070", "ex:action": "Sonicate", "ex:phase":
        "Reaction", "ex:timeUnit": "minute", "ex:timeValue": { "@type": "xsd:integer", "@value": "30" } },
    { "@id": "ex:step/reaction/S-1-1", "@type": "obo:OBI_0000070", "ex:action": "HeatChill", "ex:phase":
        "Reaction", "ex:temperatureUnit": "celsius", "ex:temperatureValue": { "@type": "xsd:integer",
        "@value": "120" }, "ex:timeUnit": "day", "ex:timeValue": { "@type": "xsd:integer", "@value": "1" } },

    { "@id": "ex:step/workup/S-1-0", "@type": "obo:OBI_0000070", "ex:action": "WashSolid", "ex:phase":
        "Workup", "obo:RO_0000057": { "@id": "ex:reagent/EtOH" } },
    { "@id": "ex:step/workup/S-1-1", "@type": "obo:OBI_0000070", "ex:action": "Dry", "ex:phase": "Workup",
        "ex:pressureUnit": "pascal", "ex:pressureValue": { "@type": "xsd:integer", "@value": "0" },
        "ex:temperatureUnit": "celsius", "ex:temperatureValue": { "@type": "xsd:integer", "@value": "120" },
        "ex:timeUnit": "day", "ex:timeValue": { "@type": "xsd:integer", "@value": "1" } },

    {
      "@id": "ex:synthesis/S-1",
      "@type": "obo:CHMO_0000410",
      "ex:componentID": "S-1",
      "ex:componentType": "glass vial",
      "obo:BFO_0000051": [
        { "@id": "ex:step/prep/S-1-0" },
        { "@id": "ex:step/prep/S-1-1" },
        { "@id": "ex:step/prep/S-1-2" },
        { "@id": "ex:step/reaction/S-1-0" },
        { "@id": "ex:step/reaction/S-1-1" },
        { "@id": "ex:step/workup/S-1-0" },
        { "@id": "ex:step/workup/S-1-1" }
      ],
      "ex:hasProduct": "MIL-88B (Fe)",
      "rdfs:label": "S-1"
    }
  ]
}
```

Listing 3.5: Representative JSON-LD excerpt from the MOF synthesis dataset for `S-1`

```
Retrieve all Prep steps with their reagents and amounts for each synthesis.
```

Listing 3.6: Example user hint for AI-assisted SPARQL generation

```
PREFIX ex: <http://example.org/>
PREFIX chmo: <http://purl.obolibrary.org/obo/CHMO_>
PREFIX obo: <http://purl.obolibrary.org/obo/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?synthesis ?step ?reagentLabel ?amount ?unit
WHERE {
  ?synthesis obo:BFO_0000051 ?step .
  ?step ex:phase "Prep" ;
        obo:RO_0000057 ?reagent ;
        ex:amountValue ?amount ;
        ex:amountUnit ?unit .
  ?reagent rdfs:label ?reagentLabel .
}
ORDER BY ?synthesis ?step
```

Listing 3.7: AI-suggested SPARQL query for Prep steps with reagent and amount information

SPARQL

Query | Result | Visualizer

Export

Search ...

?step	?unit	?amount	?reagentLabel	?synthesis
http://example.org/step/prep/S-1-0	millimole	4.0E-1	FeCl3	http://example.org/synthesis/S-1
http://example.org/step/prep/S-1-1	millimole	4.0E-1	Benzene-1,4-dicarboxylic acid	http://example.org/synthesis/S-1
http://example.org/step/prep/S-1-2	millilitre	4	DMF	http://example.org/synthesis/S-1
http://example.org/step/prep/S-2-0	millimole	4.0E-1	FeCl3·6H2O	http://example.org/synthesis/S-2
http://example.org/step/prep/S-2-1	millimole	4.0E-1	Benzene-1,4-dicarboxylic acid	http://example.org/synthesis/S-2
http://example.org/step/prep/S-2-2	millilitre	4	DMF	http://example.org/synthesis/S-2

Figure 3.1: Result after running the SPARQL query from Listing 3.7.

3.5 Knowledge Graph Visualization and Editing

After conversion of the synthesis records to JSON-LD, graph visualization becomes a direct, low-friction step in the workflow: users can open the Visualization view and inspect the same underlying RDF data as a KG, without writing additional transformation code. In this representation, synthesis runs, reagents, and process steps are shown as connected nodes, while semantic relations appear as labeled edges. Figure 3.2 illustrates this view for the mapped MOF data.

This immediate JSON-LD-to-KG transition improves usability for non-SPARQL users. Instead of interpreting nested JSON structures or triple tables, users can understand protocol structure visually: which reagent belongs to which synthesis, how Prep/Reaction/Workup steps are linked, and where key metadata is attached. As a result, semantic consistency checks can be performed faster during exploratory analysis.

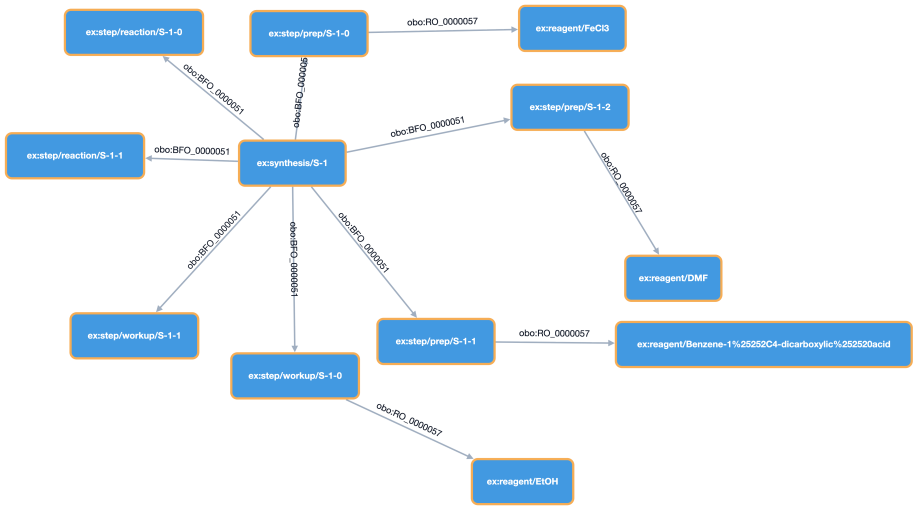


Figure 3.2: Visualization of the JSON-LD data in Listing 3.5, rendered in the Visualization dialog.

The same interface also supports lightweight editing of graph content. When a node is selected, the properties panel exposes its current predicates and values and allows users to add, edit, or delete properties directly. Node-level operations such as rename, add, and delete are also available. This means users can refine both structure and annotations in place, rather than switching between multiple tools.

Crucially, these edits are synchronized with the underlying JSON-LD/RDF model. In other words, the KG is not only a static visualization layer; it is an interactive authoring surface. For end users, this reduces modeling effort and lowers the barrier to maintaining high-quality semantic

data, because corrections can be performed where relationships are most intuitive: directly in the graph.

In addition, users can filter the graph semantically with SPARQL and visualize only the query result as a subgraph. This is particularly useful when the full dataset is large: instead of rendering all nodes, users can formulate a focused query (e.g., only Prep-related resources or only selected reagents), execute it, and inspect the returned structure directly in graph form. Hence, SPARQL-based filtering and KG visualization work together as a combined analysis workflow.

4 Conclusion and Outlook

This thesis extends MetaConfigurator with an integrated RDF authoring workflow that bridges schema-driven JSON editing and Semantic Web technologies. The implementation combines four core capabilities in one User Interface (UI): (1) transformation from JSON to JSON-LD via RML, (2) direct semantic editing through context and triple views, (3) in-browser SPARQL querying with validation support, and (4) interactive KG visualization with editing actions. In addition, AI-assisted generation of RML mappings and SPARQL queries lowers the entry barrier for non-expert users while preserving human control through review before execution.

The application chapter demonstrates this workflow on MOF synthesis data. Laboratory protocol records are transformed into ontology-grounded RDF resources (CHMO, ChEBI, and OBI), then queried and explored as a graph. This confirms the central thesis claim: schema-based data authoring and semantic data engineering can be unified in one environment, reducing tool switching and improving interoperability of scientific data.

4.1 Current Limitations

Despite these contributions, several limitations remain.

First, after the first edit in the RDF panel, the underlying JSON-LD text is regenerated (de-serialized) from the updated RDF statements. As a result, the textual representation may differ from the original input formatting and structure order, even when the represented semantics are equivalent. The root cause is that each edit currently triggers reconstruction of the whole document, rather than a localized in-place text update.

Second, AI-generated mappings and queries can still contain semantic or structural errors. Therefore, user review remains mandatory, especially for domain-critical data and ontology alignment decisions.

Third, scalability is bounded by browser-side processing constraints when working with larger graphs, especially for interactive visualization and query-result rendering. To mitigate this, two configurable limits are already integrated: `maximumTriplesToShow`, which limits how many triples are displayed in the RDF panel, and `maximumNodesToVisualize`, which limits how many nodes are

rendered in the Visualization panel. Both settings can be adjusted by the user according to dataset size and client-side performance.

4.2 Outlook

Future work should focus on four directions.

First, synchronization granularity should be improved by introducing path-aware or diff-based updates from statement edits back to the JSON-LD text view. This would preserve more of the original document layout and reduce disruptive full-document rewrites.

Second, semantic quality assurance should be strengthened through integrated SHACL validation in the authoring loop, so users can check constraints before export and publication.

Third, scalability mechanisms such as progressive graph loading, query-result size control, and graph summarization should be added to better support larger datasets.

Fourth, AI assistance should be extended with constraint-aware prompting and automated pre-execution checks to detect likely mapping and query issues earlier.

Overall, the presented extension shows that user-centered semantic authoring in scientific workflows is practical in a single application: from structured JSON input to ontology-aware querying and KG analysis.

Bibliography

[Ass26a] C. Association. *@comunica/query-sparql*. 2026. URL: https://www.npmjs.com/package/@comunica/query-sparql (cit. on p. 43).

[Ass26b] C. Association. *Comunica: Supported specifications*. 2026. URL: https://comunica.dev/docs/query/advanced/specifications/ (cit. on p. 43).

[BBB+16] A. Bandrowski, R. Brinkman, M. Brochhausen, M. H. Brush, B. Bug, M. C. Chibucos, K. Clancy, M. Courtot, D. Derom, M. Dumontier, L. Fan, J. Fostel, G. Fragoso, A. Gonzalez-Beltran, M. A. Haendel, Y. He, M. Heiskanen, T. Hernandez-Boussard, M. Jensen, Y. Lin, A. Lister, J. Malone, E. Manduchi, M. McGee, J. A. Overton, H. Parkinson, B. Peters, P. Rocca-Serra, A. Ruttenberg, S.-A. Sansone, R. H. Scheuermann, D. Schober, B. Smith, L. N. Soldatova, C. J. Stoeckert, C. F. Taylor, C. Torniai, J. A. Turner, R. Vita, P. L. Whetzel, J. Zheng. "The Ontology for Biomedical Investigations". In: *PLOS ONE* 11.4 (2016), e0154556. DOI: 10.1371/journal.pone.0154556 (cit. on p. 49).

[BHL01] T. Berners-Lee, J. Hendler, O. Lassila. "The Semantic Web". In: *Sci. Amer.* 284.5 (2001), pp. 28–37 (cit. on pp. 10, 12).

[Bra17] T. Bray. *The JavaScript Object Notation (JSON) Data Interchange Format*. RFC 8259. 2017. URL: https://www.rfc-editor.org/rfc/rfc8259 (cit. on p. 11).

[CBC+17] S. Cohen-Boulakia, K. Belhajjame, O. Collin, J. Chopard, C. Froidevaux, A. Gaignard, K. Hinsen, P. Larmande, Y. Le Bras, F. Lemoine, F. Mareuil, H. Ménager, C. Pradal, C. Blanchet. "Scientific workflows for computational reproducibility in the life sciences: Status, challenges and opportunities". In: *Future Generation Computer Systems* 75 (2017), pp. 284–298. DOI: 10.1016/j.future.2017.01.012 (cit. on p. 10).

[CBI] S. CBIR. *Protégé*. URL: https://protege.stanford.edu (cit. on p. 27).

[CG] S.-G. CG. *SPARQL-Generate*. URL: https://github.com/SPARQL-Generate (cit. on p. 28).

[CWL14] R. Cyganiak, D. Wood, M. Lanthaler. *RDF 1.1 Concepts and Abstract Syntax*. 2014. URL: https://www.w3.org/TR/rdf11-concepts/ (cit. on pp. 10, 14).

[DSC12] S. Das, S. Sundara, R. Cyganiak. *R2RML: RDB to RDF Mapping Language*. https://www.w3.org/TR/r2rml/. W3C Recommendation. 2012 (cit. on p. 29).

[DVC+14] A. Dimou, M. Vander Sande, P. Colpaert, R. Verborgh, E. Mannens, R. Van de Walle. "RML: A Generic Language for Integrated RDF Mappings of Heterogeneous Data". In: *7th Workshop on Linked Data on the Web*. 2014 (cit. on pp. 20, 29, 32).

[FLH+16] M. Franz, C. T. Lopes, G. Huck, Y. Dong, O. Sumer, G. D. Bader. "Cytoscape.js: A graph theory library for visualisation and analysis". In: *Bioinformatics* 32.2 (2016), pp. 309–311. DOI: 10.1093/bioinformatics/btv557. URL: https://js.cytoscape.org/ (cit. on p. 47).

[Fou] A. S. Foundation. *Apache Jena*. URL: https://jena.apache.org/ (cit. on p. 27).

[GOM+19] R. S. Gonçalves, M. J. O'Connor, M. Martínez-Romero, et al. "The CEDAR Workbench: An Ontology-Assisted Environment for Authoring Metadata that Describe Scientific Experiments". In: *arXiv preprint arXiv:1905.06480* (2019) (cit. on p. 27).

[Gru93] T. R. Gruber. "A Translation Approach to Portable Ontology Specifications". In: *Knowl. Acquis.* 5.2 (1993), pp. 199–220 (cit. on p. 12).

[Hav23] M. Haverbeke. *CodeMirror: In-browser Code Editor*. 2023. URL: https://codemirror.net/ (cit. on pp. 32, 43).

[HBC+21] A. Hogan, E. Blomqvist, M. Cochez, C. d'Amato, G. de Melo, C. Gutierrez, S. Kirrane, J. E. Labra Gayo, R. Navigli, S. Neumaier, A.-C. N. Ngomo, A. Polleres, S. M. Rashid, A. Rula, L. Schmelzeisen, J. Sequeda, S. Staab, A. Zimmermann. "Knowledge Graphs". In: *ACM Computing Surveys* 54.4 (2021), pp. 1–37. DOI: 10.1145/3447772 (cit. on pp. 10, 27, 28).

[HGN+14] M. Horridge, R. S. Gonçalves, C. Nyulas, T. Tudorache, M. A. Musen. "WebProtégé: A Collaborative Web-Based Platform for Editing Biomedical Ontologies". In: *Bioinformatics* (2014) (cit. on p. 27).

[ISI20] U. ISI. *Karma: A Data Integration Tool for Mapping Structured Data to RDF*. 2020 (cit. on p. 28).

[JHC+15] K. Janowicz, A. Haller, S. J. D. Cox, D. Le Phuoc, M. Lefrançois. "Why the Data Train Needs Semantic Rails". In: *AI Magazine* 36.1 (2015), pp. 5–14. DOI: 10.1609/aimag.v36i1.2568 (cit. on pp. 10, 27, 28).

[JMO+21] R. Jackson, N. Matentzoglu, J. A. Overton, R. Vita, J. P. Balhoff, P. L. Buttigieg, S. Carbon, M. Courtot, A. D. Diehl, D. M. Dooley, W. D. Duncan, N. L. Harris, M. A. Haendel, S. E. Lewis, D. A. Natale, D. Osumi-Sutherland, A. Ruttenberg, L. M. Schriml, B. Smith, C. J. Stoeckert, N. A. Vasilevsky, R. L. Walls, J. Zheng, C. J. Mungall, B. Peters. "OBO Foundry in 2021: operationalizing open data principles to evaluate ontologies". In: *Database* 2021 (2021), baab069. DOI: 10.1093/database/baab069 (cit. on p. 49).

[lin26] linkeddata. *rdflib.js Documentation*. 2026. URL: https://linkeddata.github.io/rdflib.js/ (cit. on p. 41).

[MSU+21] S. Moxon, H. Solbrig, D. R. Unni, et al. “Linked Data Modeling Language (LinkML): A General-Purpose Data Modeling Framework”. In: *Semantic Web J.* (2021) (cit. on p. 27).

[NBX+24] F. Neubauer, P. Bredl, M. Xu, K. Patel, J. Pleiss, B. Uekermann. “MetaConfigurator: A User-Friendly Tool for Editing Structured Data Files”. In: *Datenbank-Spektrum* 24 (2024), pp. 161–169. DOI: 10.1007/s13222-024-00472-7 (cit. on pp. 10, 22, 23, 27, 31).

[NEB+26] F. Neubauer, K. Endo, F. Bender, E. Ciftci, N. Hansen, S. Krause, B. Uekermann, J. Pleiss, et al. “Data-Model-Driven Management and Analysis of MOF Synthesis Data”. In: (2026) (cit. on pp. 10, 49).

[NPU25] F. Neubauer, J. Pleiss, B. Uekermann. “Data Model Creation with MetaConfigurator”. In: *Datenbanksysteme für Business, Technologie und Web (BTW 2025)*. Vol. P-361. Lecture Notes in Informatics (LNI), Proceedings. Gesellschaft für Informatik, Bonn, 2025. DOI: 10.18420/BTW2025-60. URL: https://dl.gi.de/handle/20.500.12116/45927 (cit. on pp. 22, 23, 27).

[NUP25] F. Neubauer, B. Uekermann, J. Pleiss. “AI-assisted JSON Schema Creation and Mapping”. In: *28th ACM/IEEE Int. Conf. Model Driven Eng. Languages and Systems Companion (MODELS-C)*. 2025, pp. 79–83. DOI: 10.1109/MODELS-C68889.2025.00019 (cit. on p. 24).

[Ont] Ontotext. *GraphDB: RDF Database for Knowledge Graphs*. URL: https://www.ontotext.com/products/graphdb/ (cit. on p. 27).

[Proa] O. Project. *OpenRefine*. URL: https://openrefine.org/ (cit. on p. 28).

[Prob] R. Project. *RDFLib*. URL: https://rdflib.dev/ (cit. on p. 28).

[Proc] R. R. Project. *Redland RDF Libraries*. URL: http://librdf.org/ (cit. on p. 28).

[PS13] E. Prud’hommeaux, A. Seaborne. *SPARQL 1.1 Overview*. 2013. URL: https://www.w3.org/TR/sparql11-overview/ (cit. on pp. 19, 28).

[RO26] RSC Ontologies, OBO Foundry. *Chemical Methods Ontology (CHMO)*. Ontology resource and metadata at https://obofoundry.org/ontology/chmo.html. 2026. URL: http://purl.obolibrary.org/obo/chmo.owl (cit. on p. 49).

[Sem] W. S. U. Semantic Computing Research Group. *SemTK: Semantic Toolkit*. URL: https://semtk.org/ (cit. on p. 29).

[SKL14] M. Sporny, G. Kellogg, M. Lanthaler. *JSON-LD 1.0: A JSON-based Serialization for Linked Data*. 2014. URL: https://www.w3.org/TR/json-ld/ (cit. on pp. 16, 29, 37).

[URR+26] J. F. Unger, A. Robens-Radermacher, S. M. Rosenbusch, D. Tyagi, D. Iglezakis, M. Jafarkhani. “A Platform for Validation and Verification of Models for Concrete and Concrete Structures”. In: *Computational Modelling of Concrete and Concrete Structures*. Taylor & Francis, 2026. Chap. A platform for validation and verification of models for concrete and concrete structures. DOI: 10.1201/9781003660026-32. URL: https://www.researchgate.net/publication/401659007 (cit. on p. 11).

[Vc26] R. Verborgh, contributors. *SPARQL.js*. 2026. URL: https://www.npmjs.com/package/sparqljs (cit. on p. 43).

[VKM+25] J. Villamar, M. Kelbling, H. L. More, M. Denker, T. Tetzlaff, J. Senk, et al. “Metadata practices for simulation workflows”. In: *Scientific Data* 12 (2025), p. 942. DOI: 10.1038/s41597-025-05126-1 (cit. on p. 10).

[W3C12a] W3C OWL Working Group. *OWL 2 Web Ontology Language Document Overview (Second Edition)*. 2012. URL: https://www.w3.org/TR/owl2-overview/ (cit. on p. 19).

[W3C12b] W3C OWL Working Group. *OWL 2 Web Ontology Language Primer (Second Edition)*. 2012. URL: https://www.w3.org/TR/owl2-primer/ (cit. on p. 19).

[W3C17] W3C. *Shapes Constraint Language (SHACL)*. 2017. URL: https://www.w3.org/TR/shacl/ (cit. on pp. 10, 16, 28).

[W3C23] W3C JSON-LD Community Group. *JSON-LD Playground*. Online tool maintained by the W3C JSON-LD Community Group. 2023. URL: https://json-ld.org/playground/ (cit. on p. 28).

[WAB+25] S. R. Wilkinson, M. Aloqalaa, K. Belhajjame, M. R. Crusoe, B. de Paula Kinoshita, L. Gadelha, D. Garijo, O. J. R. Gustafsson, N. Juty, S. Kanwal, F. Z. Khan, J. Köster, K. Peters-von Gehlen, L. Pouchard, R. K. Rannow, S. Soiland-Reyes, N. Soranzo, S. Sufi, Z. Sun, B. Vilne, M. A. Wouters, D. Yuen, C. Goble. “Applying the FAIR Principles to computational workflows”. In: *Scientific Data* 12.1 (2025), p. 328. DOI: 10.1038/s41597-025-04451-9 (cit. on p. 10).

[WAHD20] A. Wright, H. Andrews, B. Hutton, G. Dennis. *JSON Schema: A Media Type for Describing JSON Documents*. IETF Draft. 2020. URL: https://json-schema.org/draft/2020-12/ (cit. on p. 11).

All links were last followed on March 10, 2026.

Erklärung

Ich versichere, diese Arbeit selbstständig verfasst zu haben. Ich habe keine anderen als die angegebenen Quellen benutzt und alle wörtlich oder sinngemäß aus anderen Werken übernommene Aussagen als solche gekennzeichnet. Weder diese Arbeit noch wesentliche Teile daraus waren bisher Gegenstand eines anderen Prüfungsverfahrens. Ich habe diese Arbeit bisher weder teilweise noch vollständig veröffentlicht. Das elektronische Exemplar stimmt mit allen eingereichten Exemplaren überein.

Ort, Datum, Unterschrift